



OPEN Example dependent cost sensitive learning based selective deep ensemble model for customer credit scoring

Jin Xiao¹, Sihan Li¹, Yuhang Tian¹, Jing Huang², Xiaoyi Jiang³ & Shouyang Wang⁴

In credit scoring, data often has class-imbalanced problems. However, traditional cost-sensitive learning methods rarely consider the varying costs among samples. Moreover, previous studies have limitations, such as the lack of fit to real-world business needs and limited model interpretability. To address these issues, this paper proposes a novel example-dependent cost-sensitive learning based selective deep ensemble (ECS-SDE) model for customer credit scoring, which integrates example-dependent cost-sensitive learning with the interpretable TabNet (attentive interpretable tabular learning) and GMDH (group method of data handling) deep neural networks. Specifically, we use TabNet, which excels in handling tabular data, as the base classifier and optimize its performance on imbalanced data with an example-dependent cost loss function. Next, we design a GMDH based on an example-dependent cost-sensitive symmetric criterion to selectively deep integrate the base classifiers. This approach reduces the redundancy of base models in traditional ensemble strategies and enhances classification performance. Experimental results show that the ECS-SDE model outperforms six cost-sensitive models and five advanced deep ensemble models in overall performance for credit scoring. It shows significant advantages in the BS^+ , $Save$, and AUC metrics on four datasets. Furthermore, the ECS-SDE model provides strong interpretability, and detailed analysis reveals the key roles of various features in credit scoring.

Keywords Credit scoring, Example-dependent cost-sensitive learning, TabNet deep neural network, Selective deep ensemble, Explainable artificial intelligence

Global economic integration has created a more complex environment for financial institutions¹. In particular, the rise in financial derivatives and consumer loans has increased risks for financial institutions². Credit risk, arising from borrower defaults, is a primary concern for financial institutions³. While it is difficult to accurately predict whether a borrower will default in the future, effective credit risk scoring can significantly reduce potential default losses for financial institutions⁴. Thus, the identification of suitable measures to mitigate losses incurred by customer defaults has emerged as a critical concern in the financial industry.

Customer credit scoring is an effective tool for evaluating borrowers' credit risk. Credit scoring is commonly regarded as a binary classification task^{5–7}, which classifies borrowers into two categories: "good credit" or "poor credit." Most of the currently widely used credit scoring models are based on cost-insensitive learning methods, which aim to minimize the number of misclassifications while assuming that the cost of all misclassifications is the same⁸. However, this assumption does not fully consider the actual business objectives of financial institutions, which are to minimize operating costs⁹. For financial institutions, reducing the potential costs associated with misclassification is often more important than merely improving classification accuracy. As a result, cost-sensitive learning has emerged, aiming to minimize total classification costs by balancing management expenses and loss expenses.

Currently, many studies have applied cost-sensitive learning methods to credit scoring¹⁰, but most methods assume that the classification cost for each class (e.g., good credit vs. poor credit) is constant, which is referred to as class-dependent cost-sensitive (CCS) learning¹¹. However, the limitation of CCS is that it only focuses on the misclassification cost between different classes and primarily aims to improve the classification performance of

¹Business School, Sichuan University, Chengdu 610064, China. ²School of Public Administration, Sichuan University, Chengdu 610065, China. ³Department of Mathematics and Computer Science, University of Münster, D-48149 Münster, Germany. ⁴School of Entrepreneurship and Management, ShanghaiTech University, Shanghai 201210, China. ✉email: hansili222@126.com; syshouyangwang@126.com

the model, neglecting the need for cost minimization in the actual business operations of financial institutions¹². In real-world customer credit scoring scenarios, the economic loss to financial institutions from lending to bad customers varies, because customers have different credit limits and economic conditions¹³. To address this issue, researchers have proposed example-dependent cost-sensitive (ECS) learning. Studies have shown that, compared to CCS, ECS methods demonstrate better performance in customer credit scoring¹⁴. This is because ECS methods account for cost differences between classes as well as between samples. In customer credit scoring, ECS models can accurately estimate the economic loss caused by misclassification, taking into account the varying credit conditions and economic situations of different customers. This helps better meet the needs of financial institutions and enhances the economic benefits of credit scoring.

Lenarcik and Piasta¹⁵ first introduced the concept of ECS while improving the probabilistic rough set algorithm. Based on the stage when costs are introduced, ECS methods can be divided into three categories: introducing example-dependent costs before, during, and after model training^{8,16}. (1) Example-dependent costs introduced in pre-training methods involve adjusting sample weights according to their misclassification costs. Common methods include cost-proportionate rejection sampling (CPRS)¹⁷ and cost-proportionate over-sampling (CPOS)¹⁸. CPRS retains or rejects samples based on a probability proportional to their misclassification cost, while CPOS creates a new dataset by duplicating samples, with the frequency of duplication determined by their misclassification cost. (2) Example-dependent costs introduced during the training phase modify the loss function to directly optimize model performance. Typical models include ECS logistic regression (LR)¹⁹, ECS decision trees (DT)^{8,9}, and ECS support vector machines²⁰. (3) Example-dependent costs introduced after the training phase primarily employ a cost-sensitive Bayesian minimum risk approach^{21,22}. This approach combines the predicted probabilities from base classifiers with the example-dependent costs to minimize the overall expected risk. However, before-training approaches, which rely on the prior distribution of the training data, may lead to data bias or reduced model generalization²¹. After-training methods, in turn, depend on the base classifiers, and if they fail to effectively capture cost-sensitive information during training, optimization may be limited. In contrast, by incorporating the ECS mechanism during training, the model can more directly optimize the cost-sensitive objective, thereby improving its focus on high-cost samples. Therefore, this paper studies the ECS method that introduces example-dependent costs during the training phase.

Most of the above studies focus on improving a single classification model. However, single models are prone to overfitting, which can affect the model's generalization ability. To solve this problem, researchers have begun to enhance the performance of ECS models through ensemble learning. For example, Bahnsen et al.²³ proposed an ECS classification framework that combines ECS decision trees (CSDT) using four different ensemble methods: random forest (RF), bagging, and their variants, random patches, and pasting. The results showed that the CSDT model with the RP ensemble method produced the best performance on five datasets across four applications, including credit card fraud detection, customer churn prediction, credit scoring, and marketing. Zelenkov²⁴ used DT as base classifiers and introduced the ECS method into the AdaBoost model using three different methods: inside the exponent, outside the exponent, and both inside and outside the exponent, constructing an ECS AdaBoost ensemble model. Experiments showed that this model outperformed other ECS models on five datasets in banking marketing and insurance fraud domains. Bhargava et al.²⁵ proposed an ECS stacking ensemble framework for predicting potential tax defaulters. This framework consisted of two stages: the first stage-trained multiple cost-insensitive classifiers, and the second stage used CSDT, RF, artificial neural networks (ANN), and a bagging ensemble classifier based on CSDT as meta-models. The outputs of the first-stage models were used as inputs to train the meta-models. Experimental results showed that this framework not only outperformed traditional ECS classifiers but also significantly reduced costs.

In recent years, deep neural networks (DNN)^{26–31} have demonstrated outstanding performance in various fields, showing significant potential in credit-scoring tasks. Mehta et al.³² proposed an ECS deep neural network (ECS-DNN) by modifying the loss function to incorporate ECS. Experimental results indicated that this model had significant advantages in terms of cost savings. However, traditional DNN models typically require extensive data preprocessing when dealing with complex tabular data. In contrast, the attentive interpretable tabular deep neural network (TabNet)³³ is specifically designed for tabular data. It can be applied directly to raw data and demonstrates high prediction accuracy. As a result, researchers have attempted to introduce TabNet to credit-scoring tasks. For instance, Cai et al.³⁴ proposed a deep ensemble model for credit card fraud detection, which used TabNet as the base classifier and XGBoost for the ensemble. Experimental results showed that the proposed model outperformed the comparative models across multiple evaluation metrics. Zhang et al.³⁵ proposed a TabNet-based credit fraud detection model, which significantly outperformed traditional XGBoost and Naive Bayes algorithms. Lee et al.³⁶ used various ensemble techniques such as LightGBM, XGBoost, RF, and CatBoos to integrate TabNets, and successfully applied it to credit card default prediction. Despite the significant success of TabNet in credit scoring tasks, most existing studies focus on performance enhancement and do not consider ECS. In addition, model interpretability is particularly important in financial credit scoring. Since TabNet combines the interpretability of tree-based models with the learning ability of DNNs, it has the potential to play a greater role in this field.

However, after careful analysis, we find that the existing studies still have the following four limitations: (1) Most cost-sensitive learning-based deep learning models still adopt CCS methods, and research on ECS techniques is relatively limited¹⁴. Only one study³² has applied ECS in single DNN modeling; (2) Existing ECS ensemble models for credit scoring integrate traditional machine learning-based base classifiers, and no studies have explored ECS deep ensemble models that use deep learning models as base classifiers. In addition, existing ensemble models typically combine the predictions of all base classifiers, which may lead to redundancy. Using deep learning models as base classifiers and selecting an appropriate model subset for the ensemble, i.e., selective deep ensemble, may improve model performance; (3) Existing models that introduce the ECS mechanism during training typically adjust the loss function to account for example-dependent cost. While this adjustment reduces

misclassification costs, it may compromise performance on traditional accuracy-based metrics; (4) Current deep learning algorithms considering ECS in credit scoring are black-box models, with low transparency and poor interpretability, limiting their practical application.

To address the above limitations, this paper proposes an example-dependent cost-sensitive learning based selective deep ensemble (ECS-SDE) model for customer credit scoring. First, an example-dependent cost matrix is constructed for the raw data, and the processed dataset is randomly sampled several times to generate ECS training subsets. Second, we construct example-dependent cost-sensitive TabNet (ECS TabNet) base classifiers and train multiple differentiated base classifiers using the training subsets. Finally, we propose an example-dependent cost-sensitive GMDH (ECS GMDH) neural network that uses the selection mechanism of GMDH for the selective deep ensemble. To verify the performance of the proposed model, this paper introduces five evaluation metrics and conducts empirical analysis on four datasets. The experimental results show that, compared to three ECS models, three CCS models, and five advanced deep ensemble models, the ECS-SDE model demonstrates better overall performance in customer credit scoring and has stronger model interpretability.

The theoretical contributions of this paper are as follows: (1) We are the first to apply ECS techniques in constructing deep ensemble models for customer credit scoring by combining the interpretable TabNet and GMDH deep neural networks; (2) We introduce ECS technique to the TabNet model, proposing a new TabNet deep learning model. This model is trained by embedding an enhanced ECS-based loss function, which significantly improves its performance when dealing with imbalanced data; (3) We propose a novel example-dependent cost-sensitive symmetric criterion (ECS-SC) for the GMDH, which accounts for the cost differences between samples and aims to minimize the total cost. The ECS-SC overcomes the limitation of traditional criteria that assign equal misclassification costs to all samples, making it more feasible for the practical needs of credit scoring. Additionally, we develop an ECS-SC-based GMDH model for selective deep ensemble learning. This method resolves base model redundancy in traditional ensemble strategies, enhancing classification performance; (4) We conduct a comparative analysis using four credit-scoring datasets, comparing three ECS models, three CCS models, and five advanced deep ensemble models. The results show that the ECS-SDE model achieves superior overall performance in customer credit scoring and offers strong interpretability.

The remainder of this paper is structured as follows. Section 2 briefly reviews the relevant theoretical foundations. Section 3 provides a detailed description of the basic concept and modeling steps of the ECS-SDE model. In Sect. 4, we present the experimental design, including dataset information, experimental setup, and model evaluation metrics, and we analyze the experimental results. Finally, in Sect. 5, we present the conclusions of this paper and suggest possible future research directions.

Related works
Class dependent cost sensitive learning

In the real world, misclassification of different classes may have different consequences. In credit scoring, it is often observed that misclassifying a customer with poor credit as having good credit causes more severe economic losses than misclassifying a customer with good credit as poor credit. Therefore, many studies use CCS methods that assign different costs to the misclassification of each class. Classification costs are represented by a cost matrix, where the elements within the cost matrix are the same for all samples in the same class. Credit scoring can be represented as a binary classification problem, where samples are either in the negative class or in the positive class. To quantify the cost of misclassification, a cost matrix¹⁷ is used, as shown in Table 1:

where C_{TP} is the cost of correctly classifying a positive sample as positive. C_{FP} is the cost of incorrectly classifying a negative sample as positive. C_{FN} is the cost of wrongly classifying a positive sample as negative. C_{TN} is the cost of correctly classifying a negative sample as negative.

In recent years, CCS methods have become one of the main approaches to address class-imbalanced problems. Many researchers have combined CCS techniques with deep learning to solve the challenges of classification models on imbalanced datasets. For example, Yotsawat et al.¹⁰ proposed a class-dependent cost-sensitive neural network ensemble model (CSNNE). This model generated multiple differentiated cost-sensitive neural networks using different class weights and ensembled them through majority voting. Experiments showed that CSNNE was suitable for handling imbalanced datasets and demonstrated good performance on several credit-scoring datasets. Geng and Luo³⁷ proposed an adaptive class-dependent cost-sensitive convolutional neural network ensemble model (CSCNN). This model adaptively updated the weights of misclassification costs based on the imbalance distribution of the entire training set and local training subsets. Experimental results showed that CSCNN performed well on all evaluation metrics. Similarly, the class-dependent cost-sensitive convolutional neural network model (CCS-CNN) proposed by Vimala et al.³⁸ (2024) enhances the classification performance of minority-class samples by adjusting the classifier’s decision threshold, achieving good classification results on imbalanced datasets. Experimental results showed that the CCS-CNN method outperformed existing methods across multiple metrics.

	Actual positive	Actual negative
Predicted positive	C_{TP}	C_{FP}
Predicted negative	C_{FN}	C_{TN}

Table 1. Cost matrix.

TabNet deep neural network

TabNet³³ is a deep neural network designed for tabular data, proposed by Google in 2021. It combines the interpretability of tree models with the high predictive accuracy of DNNs. TabNet uses an end-to-end learning approach to directly learn features from raw data, reducing preprocessing time. It also provides feature importance through a sequential attention mechanism, enhancing model interpretability. TabNet has been widely applied in fields such as healthcare, insurance, and environmental studies^{5,39–41}.

TabNet constructs a sequential multi-step neural network architecture, which mainly consists of a feature transformer module and an attention transformer module. The input for each decision step is a d -dimensional feature matrix $a \in \mathbb{R}^d$. First, the initial features pass through a batch normalization (BN) layer before entering the feature transformation module. This module is composed of a fully connected layer, a BN layer, and a gated linear unit layer, which are used sequentially to process the features into more useful representations. In addition, to accelerate network convergence and stabilize the training process, *momentum* is introduced as a hyperparameter in the BN layer. This ensures that the mean and variance in the BN layer update smoothly, thereby reducing instability caused by batch size data fluctuations. In each decision step j , the features a_{j-1} processed from the previous step are input into the current step. After processing through the feature transformation module $f_j(\cdot)$, the output is split into two parts, which can be represented as follows: $[d_j, a_j] = f_j(M_{j-1} \cdot a_{j-1})$, where M_{j-1} is the mask obtained from the previous step, $d_j \in \mathbb{R}^{N_d}$ is the feature representation of the decision layer, which is output by the feature transformation module, N_d is the dimension of the decision layer features, which are used to generate the final prediction result. On the other hand, $a_j \in \mathbb{R}^{N_a}$ is the feature representation used for feature selection in the feature attention module, where N_a is the dimension of the attention layer features. The feature attention module is used to select important features. Let h_j is the combination of a fully connected layer and a BN layer. This combination performs a linear transformation and normalization on the a_{j-1} to obtain the intermediate representation $h_j(a_{j-1})$. The attention module uses the prior weight P_{j-1} from the previous step and the current step $h_j(a_{j-1})$, to obtain a sparse mask $M_j \in \mathbb{R}^d$ through the *Sparsemax* activation function:

$$M_j = \text{Sparsemax}(P_{j-1} \cdot h_j(a_{j-1})) \quad (1)$$

where *Sparsemax* is a sparse activation function used to select a small number of important features. The prior weights P_{j-1} control the frequency with which the model selects features. These weights are calculated using the previous masks and a relaxation factor *gamma* as follows: $P_{j-1} = \prod_{k=1}^{j-1} (\text{gamma} - M_k)$, where k is the step number ($k = 1, 2, \dots, j-1$), and *gamma* is a hyperparameter that controls the flexibility of feature selection. When *gamma* = 1, the model enforces the use of a feature in each step. As *gamma* increases, the likelihood of reusing the same feature across multiple steps increases, reducing the constraints on feature selection at each step, and thereby enhancing the model's flexibility. Then, the new mask M_j and the new feature a_j generated at the j -th step will be passed to the next decision step. This process is repeated until the preset number of steps N_{step} is reached or a stopping condition is met.

Based on the feature masks at each step, the local importance score for each feature can be obtained. The local importance score $S_{i,j}$ for the i -th feature at the j -th step is expressed as: $S_{i,j} = \sum_{j=1}^{N_{step}} \eta_j M_{i,j}$, where $M_{i,j}$ is the mask value for the i -th feature at the j -th step, and η_j is the weight factor for the j -th step. Finally, by aggregating the masks and weight factors from all steps, the global importance score for the i -th feature is obtained:

$$S_i = \frac{\sum_{j=1}^{N_{step}} \eta_j M_{i,j}}{\sum_{i=1}^d \sum_{j=1}^{N_{step}} \eta_j M_{i,j}} \quad (2)$$

at the same time, by aggregating the outputs of all decision layers, the final decision output d_{final} is expressed as: $d_{final} = \sum_{j=1}^{N_{step}} \text{ReLU}(d_j)$, where *ReLU* is the activation function used to process the decision layer outputs. Finally, the aggregated decision layer output d_{final} is mapped to the model's output space through a fully connected layer to generate the final prediction result. In binary classification problems, TabNet typically uses the binary cross-entropy loss function for training, which is expressed as:

$$\text{Loss}(y, \hat{y}) = - \sum (y * \log(\hat{y}) + (1 - y) * \log(1 - \hat{y})) \quad (3)$$

where y is the true value, and $\hat{y}$ is the predicted value.

GMDH neural network

GMDH neural network is a self-organizing inductive modeling technique⁴², commonly used for modeling and identifying complex systems. Let $X = (x_1, x_2, \dots, x_n)$ and y represent the input and output variables, respectively. The modeling process of GMDH is as follows:

First, the input dataset D_{input} is randomly divided into a learning set A and a selection set B . Typically, a discrete Kolmogorov-Gabor (K-G) polynomial is used to establish the general relationship between the input and output variables:

$$Y = w_0 + \sum_{i=1}^N w_i x_i + \sum_{i=1}^N \sum_{j=1}^N w_{ij} x_i x_j + \dots \quad (4)$$

Where w_0, w_i, w_{ij} are the weights. Next, an initial input model set $V = \{v_1 = x_1, v_2 = x_2, \dots, v_n = x_n\}$ is created. These initial models in V are then combined pairwise using a transfer function $f(\cdot)$, generating the first layer of $n_1 = C_n^2$ intermediate candidate models in total. Then, the ordinary least squares (OLS) method is used to estimate the parameters of candidate models on set A , and the external criterion values of candidate models are calculated on set B . The candidate models are ranked based on these criterion values, and the optimal $F_1 (\leq C_n^2)$ models are selected. To avoid losing important information too early, the initial models are included in the intermediate candidate model set for each layer⁴³. That is, the selected candidate models are combined with the n initial models and once again pairwise combined using the transfer function, generating the second layer of $F_2 = C_{F_1+n}^2$ candidate models. From this, the optimal F_2 models are selected. Finally, this process is repeated layer by layer to generate intermediate candidate models. The process continues until a termination criterion is met, i.e., the external criterion value initially decreases and then increases as the complexity of the candidate models increases⁴⁴. When the external criterion value reaches its minimum, the optimal complexity model Y^* with m layers is obtained. The structure of the GMDH neural network is shown in Fig. 1.

The most commonly used external criterion for GMDH is the symmetric regularity criterion (SRC). This criterion primarily evaluates the fitting accuracy of the model. Its mathematical expression is as follows:

$$d^2(D_{input}) = \Delta^2(A) + \Delta^2(B) \\ = \sum_{x \in B} (y_B - \hat{y}_B(A))^2 + \sum_{x \in A} (y_A - \hat{y}_A(B))^2 \quad (5)$$

where y_B is the actual output of set B , and $\hat{y}_B(A)$ is the predicted output of set B by the model constructed on set A . Similarly, y_A is the actual output of set A , and $\hat{y}_A(B)$ is the predicted output of set A by the model constructed on set B . $\Delta^2(A)$ is the error on set B by the model constructed on set A , $\Delta^2(B)$ is the error on set A by the model constructed on set B , and $d^2(D_{input})$ is the total error on D_{input} .

However, in the SRC, all samples are assigned the same misclassification cost. In credit scoring, in contrast, different classes often have different misclassification costs. Therefore, in our previous research⁴⁵, we combined CCS with SRC and proposed a class-dependent cost-sensitive symmetric regularity criterion (CS-SRC):

$$\text{Cost}(D_{input}) = \text{Cost}(A) + \text{Cost}(B) \quad (6)$$

$$\text{Cost}(A) = \sum_{x=1}^{n_{11}} (y_B - \hat{y}_B(A))^2 + \sum_{x=1}^{n_{12}} (y_B - \hat{y}_B(A))^2 \quad (7)$$

$$\text{Cost}(B) = \sum_{x=1}^{n_{21}} \varepsilon (y_A - \hat{y}_A(B))^2 + \sum_{x=1}^{n_{22}} (y_A - \hat{y}_A(B))^2 \quad (8)$$

where n_{11} and n_{12} are the numbers of positive and negative samples in subset B , n_{21} and n_{22} are the numbers of positive and negative samples in set A , respectively. Assume that the misclassification cost for each negative sample is 1, while the misclassification cost for positive samples is ε . $\text{Cost}(A)$ is the total misclassification cost of set B by the model constructed on set A , $\text{Cost}(B)$ is the total misclassification cost of set A by the model constructed on set B , $\text{Cost}(D_{input})$ is the total misclassification cost on set D_{input} .

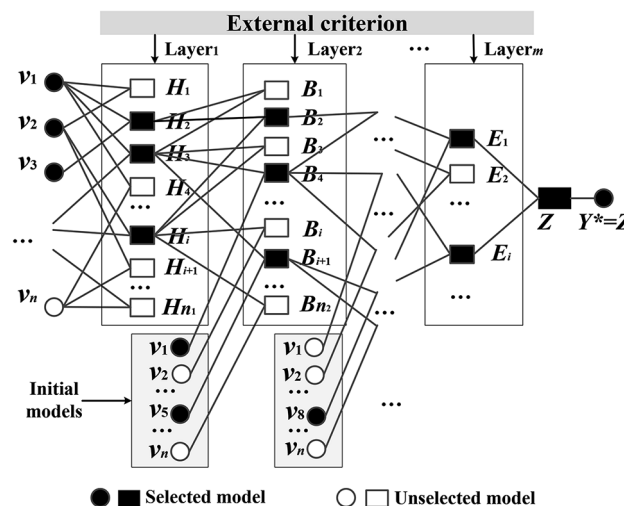


Fig. 1. The process of finding the optimal complexity model in the GMDH neural network.

Methods

Basic framework

Existing credit scoring models often use traditional CCS techniques. However, these methods fail to account for cost differences between samples and rarely consider practical business needs or model interpretability. To address these issues, this paper proposes an ECS-SDE model for customer credit scoring.

Let $D = \{(x_i, y_i)\}_{i=1}^N$ be a dataset containing N samples, where $x_i \in \mathbb{R}^n$ is an n -dimensional vector and $y_i \in \{0, 1\}$ is the class label of x_i . D_{maj} and D_{min} are the majority and minority class samples in D , respectively. The modeling process of the ECS-SDE model mainly consists of three phases:

Phase I: construction of the example-dependent cost matrix and ECS training subset

First, based on the example-dependent cost matrix in the credit scoring domain, this paper calculates the cost matrix C_i for each sample x_i . Next, the cost matrix is added to the dataset D to create the new dataset $D' = \{(x_i, C_i, y_i)\}_{i=1}^N$. Then, D' is randomly divided into a training set D_{train} and a test set D_{test} . Finally, several random samplings are performed on D_{train} to generate the ECS training subset D_{sub} .

Phase II: training of ECS TabNet base classifiers

First, this paper constructs the ECS TabNet base classifier by embedding a new loss function. Then, M differentiated ECS TabNet base classifiers, denoted as $\{T_1, T_2, \dots, T_M\}$, are trained on the ECS training subset D_{sub} . The prediction result of the j -th base classifier T_j on the j -th ECS training subset is denoted as $\hat{y}_j = \{\hat{y}_{ij}\}_{i=1}^N$ ($j = 1, 2, \dots, M$). Thus, the prediction results of all base classifiers on the training subsets are $\hat{y}_1, \hat{y}_2, \dots, \hat{y}_M$.

Phase III: design of an ECS GMDH for the selective deep ensemble

First, this paper proposes a new ECS-SC external criterion to construct the ECS GMDH neural network. Then, the ECS GMDH is used to perform a selective deep ensemble on the prediction results of the M ECS TabNet base classifiers, ultimately yielding the credit-scoring result. The framework of ECS-SDE is shown in Fig. 2.

Construction of the example dependent cost matrix and ECS training subset

In ECS learning, different samples correspond to different cost matrices. For customer credit scoring, this paper uses the example-dependent cost matrix proposed by Bahnsen et al.¹⁹ (Table 2) and applies the corresponding calculation formula (Eq. 9) to derive the cost matrix for all samples.

$$\text{Cost}(D') = \sum_i^N \text{Cost}(y_i, \hat{y}_i) \quad (9)$$

where $\text{Cost}(y_i, \hat{y}_i) = y_i(\hat{y}_i C_{TP_i} + (1 - \hat{y}_i) C_{FN_i}) + (1 - y_i)(\hat{y}_i C_{FP_i} + (1 - \hat{y}_i) C_{TN_i})$, where y_i is the actual output of a sample x_i , and $\hat{y}_i$ is the predicted output of a sample x_i , $\text{Cost}(D')$ is the total misclassification cost for all samples. When $y_i = 1$, the cost is $\hat{y}_i C_{TP_i} + (1 - \hat{y}_i) C_{FN_i}$. When $y_i = 0$, the cost is $\hat{y}_i C_{FP_i} + (1 - \hat{y}_i) C_{TN_i}$.

Next, the dataset D' is randomly split into a training set D_{train} and a test set D_{test} . Finally, multiple random samplings are performed on D_{train} to generate ECS training subsets D_{sub} .

Training of ECS TabNet base classifiers

Traditional TabNet deep neural networks treat all samples equally during training, which may lead to underestimating the importance of minority-class samples, especially in class-imbalanced problems. To address this, we replace the traditional loss function (Eq. 3) with an enhanced example-dependent cost function (Eq. 9), resulting in an improved loss function.

Specifically, to address class imbalance, this paper considers the importance of minority-class samples in credit scoring. According to Elkan¹⁸, in credit scoring, misclassification costs for minority-class samples could be up to 5 times higher than that for majority-class samples. Therefore, the new loss function calculates example-dependent costs separately for both classes, multiplying the cost for minority-class samples by 5 to place greater emphasis on them during training. The new loss function is as follows:

$$Loss_{cost}(y_i, \hat{y}_i) = \sum_{x \in D_{maj}} \text{Cost}_{maj}(y_i, \hat{y}_i) + \sum_{x \in D_{min}} \text{Cost}_{min}(y_i, \hat{y}_i) \quad (10)$$

$$\text{Cost}_{maj}(y_i, \hat{y}_i) = \left(y_i \left(\hat{y}_i C_{TP_i}^{maj} + (1 - \hat{y}_i) C_{FN_i}^{maj} \right) + (1 - y_i) \left(\hat{y}_i C_{FP_i}^{maj} + (1 - \hat{y}_i) C_{TN_i}^{maj} \right) \right) \quad (11)$$

$$\text{Cost}_{min}(y_i, \hat{y}_i) = 5 * \left(y_i \left(\hat{y}_i C_{TP_i}^{min} + (1 - \hat{y}_i) C_{FN_i}^{min} \right) + (1 - y_i) \left(\hat{y}_i C_{FP_i}^{min} + (1 - \hat{y}_i) C_{TN_i}^{min} \right) \right) \quad (12)$$

where $[C_{FP_i}^{maj}, C_{FN_i}^{maj}, C_{TP_i}^{maj}, C_{TN_i}^{maj}]$ is the cost matrix for majority class samples, and $\text{Cost}_{maj}(\cdot)$ is the misclassification cost generated by majority class samples. Similarly, $[C_{FP_i}^{min}, C_{FN_i}^{min}, C_{TP_i}^{min}, C_{TN_i}^{min}]$ is the cost matrix for minority class samples, and $\text{Cost}_{min}(\cdot)$ is the misclassification cost generated by minority class samples. $Loss_{cost}(\cdot)$ is the total misclassification cost.

Next, we build the ECS TabNet classifier by embedding the new loss function. We train on M ECS training subsets D_{sub} to generate M differentiated ECS TabNets, denoted as $\{T_1, T_2, \dots, T_M\}$. Let the prediction results

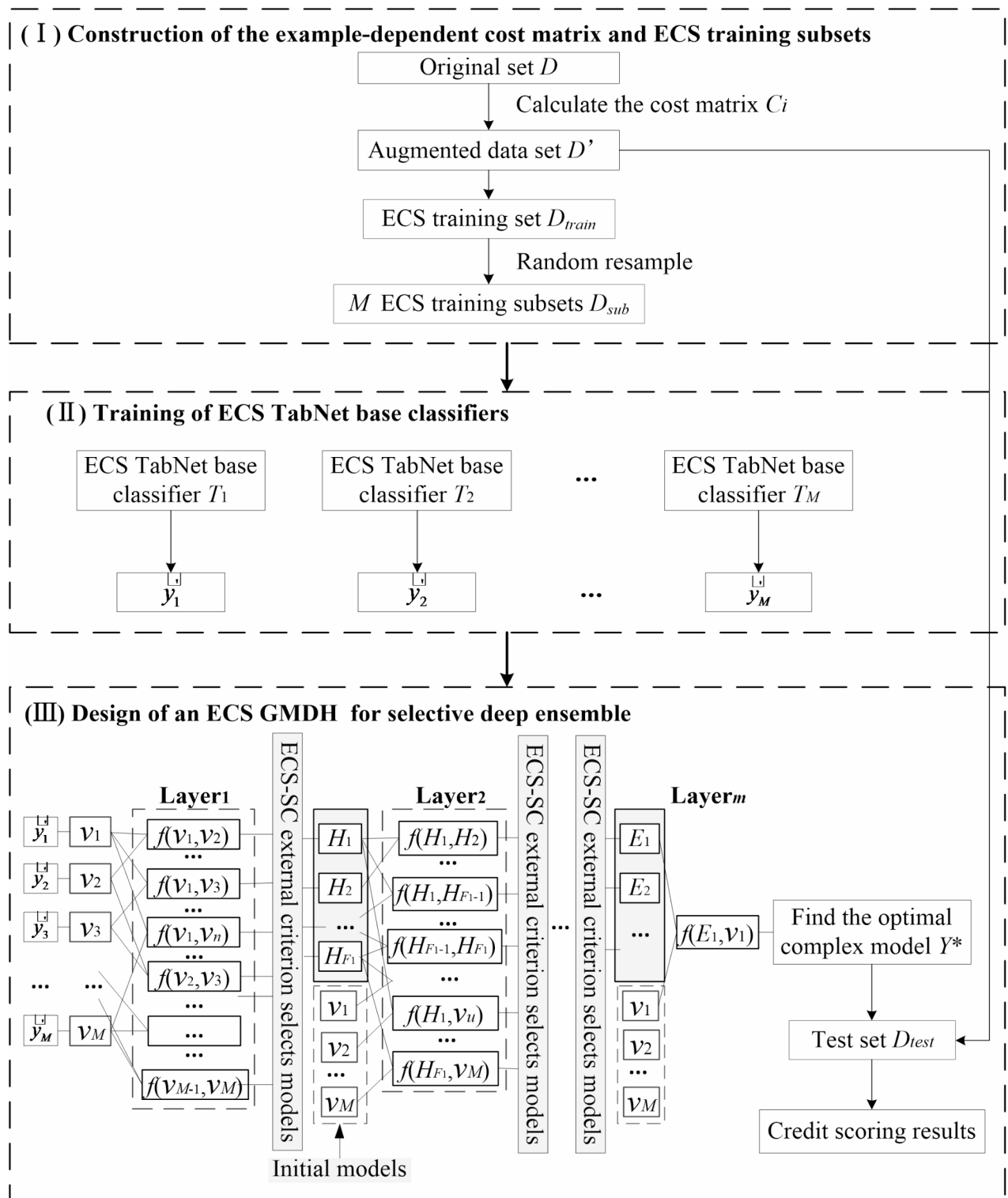


Fig. 2. Framework of ECS-SDE model.

of the j -th base classifier T_j on the j -th training subset be $\hat{y}'_j = \{\hat{y}'_{ji}\}_{i=1}^N$ ($j = 1, 2, \dots, M$). Thus, the prediction results of all base classifiers on training subsets are denoted as $\hat{y}'_1, \hat{y}'_2, \dots, \hat{y}'_M$.

Design of an ECS GMDH for selective deep ensemble

First, let the predicted outputs of the base classifiers be $\hat{Y}' = (\hat{y}'_1, \hat{y}'_2, \dots, \hat{y}'_M)$ and the actual outputs be y , which will serve as the input and output vectors for the ECS GMDH neural network, respectively. This forms a new input dataset $D_{input} = (\hat{Y}', y)$. Then, D_{input} is randomly split into a model learning set A and a model

	Actual positive	Actual negative
Predicted positive	$C_{TP_i} = 0$	$C_{FP_i} = r_i + C_{FP}^a$
Predicted negative	$C_{FN_i} = Cl_i * L_{gd}$	$C_{TN_i} = 0$

Table 2. Example-dependent cost matrix. For the i -th customer sample x_i in the dataset D , its cost matrix is $C_i = [C_{FP_i}, C_{FN_i}, C_{TP_i}, C_{TN_i}]$, where C_{TP_i} is the cost of correctly classifying a positive class as positive, C_{FP_i} is the cost of misclassifying a negative class as positive, C_{FN_i} is the cost of misclassifying a positive class as negative, and C_{TN_i} is the cost of correctly classifying a negative class as negative. Specifically, C_{FP_i} is composed of the sum of r_i and C_{FP}^a , where r_i is the loss from losing a quality customer. r_i can be calculated using the time value formula: $r_i = PV(A(Cl_i, int_{r_i}, l_i), int_{cf}, l_i) - Cl_i$, where A is the customer's monthly repayment amount, PV is the present value of monthly repayments, int_{r_i} is the loan interest rate, l_i is the loan term, and int_{cf} is the cost of capital. The customer's credit limit Cl_i is calculated as follows: $Cl_i = \min(q \cdot Inc_i, Cl_{max}, Cl_{max}(debt_i))$, where Inc_i is the customer's income, q is a parameter that defines the maximum credit limit Cl_i as a function of the income Inc_i and $debt_i$ is the debt ratio. The maximum total credit limit $Cl_{max}(debt_i)$ can be calculated as: $Cl_{max}(debt_i) = PV(Inc_i \cdot P_m(debt_i), int_{r_i}, l_i)$, where $P_m(debt_i) = \min(A(q \cdot Inc_i, int_{r_i}, l_i) / Inc_i(1 - debt_i))$ is the current debt ratio. The assumption that the financial institution does not retain the idle capital, C_{FP}^a is the potential loss from rejecting a quality customer, and is calculated as: $C_{FP}^a = -\bar{r} \cdot \pi_0 + \bar{Cl} \cdot L_{gd} \cdot \pi_1$, where $\bar{Cl}$ is the average credit limit in the market, $\bar{r}$ is the average profit margin, L_{gd} is a loss due to bad debt as a proportion of the credit line, and π_1 and π_0 are the prior probabilities of potential customers defaulting or repaying the loan, respectively. Additionally, C_{FN_i} is the product of Cl_i and L_{gd} . It is generally assumed¹⁸ that the cost of misclassification should be greater than the cost of correct classification, i.e., $C_{FN_i} > C_{TP_i}$ and $C_{FP_i} > C_{TN_i}$, and the cost of correct classification is zero, i.e., $C_{TP_i} = C_{TN_i} = 0$. Based on the above cost matrix, the augmented feature vector for each sample can be obtained as $[x_i, C_i]$. The dataset D can then be expanded to a new dataset $D' = \{(x_i, C_i, y_i)\}_{i=1}^N$, where the overall misclassification cost for the N samples in D' is calculated as follows²¹:

selection set B . Next, an initial model set $V = \{v_1 = \hat{y}'_1, v_2 = \hat{y}'_2, \dots, v_n = \hat{y}'_n\}$ is created. All initial models in the set V are pairwise combined using the transfer function $f(v_i, v_j) = w_0 + w_1 v_i + w_2 v_j + w_3 v_i v_j$ (for $i, j = 1, 2, \dots, M$ with $i \neq j$) to generate the first layer of intermediate candidate models. It is important to note that in the real world of credit scoring, due to operational cost constraints, companies can only manage a portion of customers that are most likely to reduce operational costs. Therefore, the question for companies is how much money can be saved with the help of the model.

To achieve this goal, inspired by previous research⁴⁵, we introduce the example-dependent cost function (Eq. 9) into the external criteria of GMDH and propose a novel criterion, the example-dependent cost-sensitive symmetric criterion (ECS-SC). Traditional SRC criterion selects models by minimizing overall classification error, assuming equal misclassification costs for all samples. In contrast, ECS-SC accounts for cost differences between samples and optimizes total cost, better aligning with the practical needs of financial institutions. Specifically, ECS-SC calculates the example-dependent costs for majority and minority-class samples separately, assigning higher weights to minority-class samples (we still set the weight to 5) to emphasize their importance in model selection. The ECS-SC is defined as follows:

$$\text{Cost}(D_{input}) = \text{Cost}(A) + \text{Cost}(B) \quad (13)$$

$$\text{Cost}(A) = \sum_{x \in B_{maj}} \text{Cost}_{maj}(A) + \sum_{x \in B_{min}} \text{Cost}_{min}(A) \quad (14)$$

$$\text{Cost}(B) = \sum_{x \in A_{maj}} \text{Cost}_{maj}(B) + \sum_{x \in A_{min}} \text{Cost}_{min}(B) \quad (15)$$

$$\text{Cost}_{maj}(A) = \left(y_B^i \left(\hat{y}_B^i(A) C_{TP_i}^{maj} + (1 - \hat{y}_B^i(A)) C_{FN_i}^{maj} \right) + (1 - y_B^i) \left(\hat{y}_B^i(A) C_{FP_i}^{maj} + (1 - \hat{y}_B^i(A)) C_{TN_i}^{maj} \right) \right) \quad (16)$$

$$\text{Cost}_{min}(A) = 5 * \left(y_B^i \left(\hat{y}_B^i(A) C_{TP_i}^{min} + (1 - \hat{y}_B^i(A)) C_{FN_i}^{min} \right) + (1 - y_B^i) \left(\hat{y}_B^i(A) C_{FP_i}^{min} + (1 - \hat{y}_B^i(A)) C_{TN_i}^{min} \right) \right) \quad (17)$$

$$\text{Cost}_{maj}(B) = \left(y_A^i \left(\hat{y}_A^i(B) C_{TP_i}^{maj} + (1 - \hat{y}_A^i(B)) C_{FN_i}^{maj} \right) + (1 - y_A^i) \left(\hat{y}_A^i(B) C_{FP_i}^{maj} + (1 - \hat{y}_A^i(B)) C_{TN_i}^{maj} \right) \right) \quad (18)$$

$$\text{Cost}_{min}(B) = 5 * \left(y_A^i \left(\hat{y}_A^i(B) C_{TP_i}^{min} + (1 - \hat{y}_A^i(B)) C_{FN_i}^{min} \right) + (1 - y_A^i) \left(\hat{y}_A^i(B) C_{FP_i}^{min} + (1 - \hat{y}_A^i(B)) C_{TN_i}^{min} \right) \right) \quad (19)$$

where y_B^i is the actual output of the i -th sample x_i on set B , and $\hat{y}_B^i(A)$ is the predicted output of the i -th sample x_i on set B by the model constructed on set A . Similarly, y_A^i is the actual output of the i -th sample x_i on set A , and $\hat{y}_A^i(B)$ is the predicted output of the i -th sample x_i on set A by the model constructed on set B . A_{maj} and A_{min} are the majority and minority class samples in set A , respectively, and B_{maj} and B_{min} are the majority and minority class samples in set B , respectively. $\text{Cost}_{maj}(A)$, $\text{Cost}_{min}(A)$ and $\text{Cost}(A)$ are the misclassification costs of majority class samples, minority class samples, and the overall misclassification cost, respectively, when the model constructed in set A is applied to set B . $\text{Cost}_{maj}(B)$, $\text{Cost}_{min}(B)$ and

$\text{Cost}(B)$ represent the misclassification costs of majority class samples, minority class samples, and the overall misclassification cost, respectively, when the model constructed in set B is applied to set A .

It should be noted that the traditional GMDH typically uses the OLS method to estimate the parameters of candidate models. However, as the number of layers in the GMDH network increases, the correlations between input variables also increase, which may lead to multicollinearity issues, thereby affecting the model performance⁴⁰. To address this issue, we introduce an $L2$ regularization term, which compresses some of the highly correlated parameters to near zero, effectively suppressing model overfitting and mitigating the effects of multicollinearity. The expression is as follows:

$$J(\hat{w}) = J_{LS}(\hat{w}) + \lambda \|\hat{w}\|_2^2 \quad (20)$$

where $J_{LS}(\hat{w})$ is the sum of squared errors of the model parameters estimated by the OLS method, $\|\hat{w}\|_2^2$ is $L2$ norm, and λ is a constant used to adjust the relative strength between $J_{LS}(\hat{w})$ and $\|\hat{w}\|_2^2$. Specifically, as λ increases, some of the less important model parameters are compressed towards zero, leading the model to produce sparser solutions, thereby reducing the model complexity.

Then, based on Eq. 13, the ECS-SC external criterion value for the first layer of intermediate candidate models is calculated and ranked. The top F_1 models with the best external criterion values are selected. Next, the selected F_1 candidate models, along with the initial models, are combined again using the transfer function $f(\cdot)$ in pairs to generate the next layer of candidate models. Finally, the process is repeated until the ECS-SC external criterion value reaches its minimum, obtaining the optimal complexity model Y^* .

Modeling process

The detailed modeling process of the ECS-SDE model is as follows:

Phase I: construction of the example-dependent cost matrix and ECS training subset

Step 1: For each sample x_i , we calculate its corresponding cost matrix $C_i = [C_{FP_i}, C_{FN_i}, C_{TP_i}, C_{TN_i}]$ and expand the original dataset $D = \{(x_i, y_i)\}_{i=1}^N$ into a new dataset $D' = \{(x_i, C_i, y_i)\}_{i=1}^N$. Then, we randomly divide D' into a training set $D_{train} = \{(x_{train}, C_{train}, y_{train})\}$ and a test set $D_{test} = \{(x_{test}, C_{test}, y_{test})\}$;

Step 2: Multiple random samplings are performed on the training set D_{train} to generate M ECS training subsets D_{sub} ;

Phase II: training of ECS TabNet base classifiers

Step 3: ECS TabNet base classifiers are constructed, and M ECS training subsets D_{sub} are used for training. This results in M differentiated ECS TabNets, denoted as $\{T_1, T_2, \dots, T_M\}$;

Phase III: design of an ECS GMDH for selective deep ensemble

Step 4: ECS-SC external criterion is constructed, and the ECS GMDH neural network is built based on this criterion;

Step 5: Prediction results of the M ECS TabNets are taken as inputs for the ECS GMD. The ECS-SC external criterion values for each layer of candidate models are calculated based on Eq. 13;

Step 6: The process continues until the external criterion value reaches its minimum, obtaining the optimal complexity model Y^* and achieving selective deep ensemble predictions.

The modeling flowchart of the ECS-SDE model is shown in Fig. 3.

Input: Dataset D , number of base classifiers M , hyperparameters for ECS TabNet including attention layer dimension N_a , decision layer dimension N_d , number of steps N_{step} , relaxation factor γ for controlling feature selection mask, and μ for controlling feature selection update process; hyperparameter λ for ECS GMDH model complexity regulation.

Output: Final trained optimal complexity model Y^* ;

- 1: Calculate the example-dependent cost matrix C_i , and expand the dataset D into D' ;
- 2: Divide D' into the training set D_{train} and test set D_{test} ;
- 3: Initialize the ECS TabNet base classifiers;
- 4: For $u = 1$ to M :
 - 5: Randomly sample the ECS training subset D_{sub} from D_{train} ;
 - 6: Train the ECS TabNet base classifier using D_{sub} , and obtain T_u ;
- 7: Obtain the set of base classifiers $\{T_1, T_2, \dots, T_M\}$ and the corresponding prediction outputs $\{\hat{Y}_1, \hat{Y}_2, \dots, \hat{Y}_M\}$;
- 8: Use the predictions from the M base classifiers as inputs for the ECS GMDH neural network;
- 9: Calculate the ECS-SC external criterion values for each layer of intermediate candidate models based on Eq. 13;
- 10: Repeat the process until the ECS-SC external criterion value reaches the minimum, obtaining the optimal complexity model Y^* , which is the ECS-SDE model.

Algorithm 1. ECS-SDE Mode.

Results and analysis

This section presents comparative experiments to evaluate the effectiveness of the proposed ECS-SDE model. Section 4.1 to 4.3 introduce the datasets, experimental settings, and evaluation metrics. In Sect. 4.4, the ECS-SDE model's performance is compared with three ECS models, three CCS models, and five deep ensemble models. Section 4.5 compares the computation time of ECS-SDE with five deep ensemble models. Section 4.6 presents ablation experiments to assess the impact of ECS TabNet and ECS GMDH on model performance. Section 4.7 analyzes the interpretability of the ECS-SDE model, and Sect. 4.8 conducts sensitivity analysis on ECS TabNet parameters, the number of base classifiers, and ECS GMDH parameters.

Datasets

This paper evaluates the model using four credit-scoring datasets, including the *IEEE-CIS Fraud Detection* (IEEE) dataset from the Kaggle competition. This dataset, which aims to predict online transaction fraud, contains 151 features and 1 binary label. The data is divided into transaction and identity information, covering aspects such as transaction amount, payment card details, and digital signatures. The *Give Me Some Credit* (GMSC) dataset, also from Kaggle, is used to predict the likelihood of a customer experiencing financial distress within two years, helping determine loan issuance. It contains 10 features and 1 binary label, with key features including credit utilization rate, debt ratio, and monthly income et al. The *Default of Credit Card Clients* (DCCC) dataset, sourced from the UCI public database, records customer credit card payment history in Taiwan from April to September 2005. It contains 23 features and 1 binary label, with features related to credit limit, age,

Datasets	Number of samples	Number of features	IR
IEEE	589,099	151	28.57
GMSC	112,915	10	13.83
DCCC	30,000	23	4.52
PAKDD	38,938	20	4.03

Table 3. Dataset description.

repayment history, and more. The 2009 Pacific-Asia Knowledge Discovery and Data Mining Conference (PAKDD) dataset includes credit data from a Brazilian financial institution, collected between 2003 and 2008. It contains 20 features and 1 binary label, with attributes such as customer age, personal net income, gender et al. Table 3 provides the basic information of the four credit-scoring datasets, where the imbalanced ratio (IR) is defined as the ratio of majority class (good credit) samples to minority class (bad credit) samples. A higher IR value indicates a greater imbalance in the class distribution. The datasets used in this paper were preprocessed as described in the literature^{14,47}. The “Data availability” section at the end of the paper provides details on how to obtain these datasets, with clickable links for accessing specific acquisition information. The GMSC dataset is used as a case study, and a detailed feature description is included in Appendix A for a deeper analysis of the model’s interpretability.

Experimental setup

In this experiment, we used four credit-scoring datasets, and the following steps were performed for each dataset. First, the augmented dataset D' is randomly divided into a training set and a test set in a 6:4 ratio. In the training set, 90% of the samples are used to train the model, and the remaining 10% are used for hyperparameter optimization. To reduce the randomness of the results, we repeat the entire experiment 10 times and calculate the average of the results for subsequent analysis and model performance comparison. Additionally, the credit scoring example-dependent cost matrix is shown in Table 2, where the personal income Inc_i can be directly obtained from the dataset, and the debt ratio $debt_i$ can be indirectly calculated based on information such as income and credit limit in the dataset. Other parameters, such as the market average credit limit $\bar{CL}$, the average profit margin $\bar{r}$, and the loan term l_i , are set based on the research by Bahnsen et al.¹⁹.

In the model comparison, this paper compares the proposed ECS-SDE model with other models that use cost-sensitive techniques, including three ECS models and three CCS models. Given that the ECS-SDE model is a deep ensemble framework based on ECS, a review of the literature reveals that the latest advancements in ECS-based models primarily focus on traditional ensemble models and deep learning models. Therefore, the three ECS models selected include: the example-dependent cost-sensitive AdaBoost model using the outside exponent method (ECS-AdaBoost) proposed by Zelenkov¹⁷, the example-dependent cost-sensitive deep neural network (ECS-DNN) proposed by Mehta et al.³², and the example-dependent cost-sensitive stacking ensemble framework (ECS-Stacking) proposed by Bhargava et al.²⁵. Next, the three CCS models are as follows: the class-dependent cost-sensitive neural network ensemble model (CSNNE) proposed by Yotsawat et al.¹⁰ and the class-dependent cost-sensitive convolutional neural network ensemble model (CSCNN) proposed by Geng and Luo³⁷, and the class-dependent cost-sensitive convolutional neural network (CNN) model (CCS-CNN) proposed by Vimala et al.³⁸.

To further evaluate the performance of the ECS-SDE model, this paper compares it with five advanced deep ensemble models: the deep ensemble model based on long short-term memory (LSTM) and gated recurrent unit (GRU) neural networks (LSTM-GRU-ANN) proposed by Forough and Momtazi⁴⁹, the deep ensemble model based on deep recurrent neural networks (LSTM-GRU-MLP) proposed by Mienye and Sun⁵⁰, the deep ensemble model based on CNNs and bidirectional long short-term memory (BiLSTM) networks (CNN-BiLSTM) proposed by Haghighi and Omranpour⁵¹, as well as the deep ensemble models based on CNN and BiLSTM (BiLSTM-CNN), and on CNN, BiLSTM, and Transformer (BiLSTM-Trans-CNN), both proposed by Wang et al.⁵². The parameter settings for the comparative models are shown in Table 4.

Model	Parameter settings
ECS-AdaBoost	Base classifier is a decision tree, the number of classifiers is set to 20, and the boosting algorithm used is SAMME.R.
ECS-DNN	ECS-DNN model parameter settings follow the study by Mehta et al. ³² .
ECS-Stacking	Base classifiers used include various cost-insensitive models, such as KNN, XGBoost, RF, LR, ANN, and AdaBoost. The meta-model uses a bagging classifier based on ECS decision trees. Specific parameter settings referenced from Bhargava et al. ²⁵
CSNNE	Base classifier is an ANN with 2 hidden layers, using ReLU activation for the hidden layers and Softmax for the output. The Adam optimizer is applied, with a batch size of 64 and 300 epochs. The ensemble includes 9 base classifiers, with majority voting as the strategy. Parameter settings are based on Yotsawat et al. ¹⁰ .
CSCNN	Ensemble includes 4 base CNN classifiers, each with 3 hidden layers (32, 32, and 64 neurons), ReLU activation for hidden layers, and Sigmoid for the output. The Adam optimizer is used with a batch size of 512, 100 epochs, and a dropout rate of 0.5. Bagging is employed as the ensemble strategy, with parameters based on the study by Geng and Luo ³⁷ .
CCS-CNN	CNN has 3 hidden layers (32, 64, and 64 neurons) with ReLU activation and a Sigmoid output layer. Adam optimizer is used with a batch size of 128, 100 epochs, and a dropout rate of 0.5. Decision threshold is optimized through grid search, set to 0.35. Parameters are based on Vimala et al. ³⁸ .
LSTM-GRU-ANN	Base classifiers are LSTM and GRU models with Tanh activation for the hidden layers and Sigmoid for the output. Ensemble strategy uses an ANN with ReLU activation for the hidden layers and Sigmoid for the output layer. Parameters are based on Forough and Momtazi ⁴⁹ .
LSTM-GRU-MLP	Base classifiers are LSTM and GRU models with Tanh activation for the hidden layers and Sigmoid for the output. Ensemble strategy uses a multi-layer perceptron. Parameters are based on Mienye and Sun ⁵⁰ .
CNN- BLSTM	Base classifier is CNN with 10 base classifiers, using ReLU for hidden layers and Sigmoid for the output. The Adam optimizer is employed. The ensemble strategy uses BiLSTM, with Tanh for hidden layers and Sigmoid for the output. Parameters are based on Haghighi and Omranpour ⁵¹ .
BiLSTM-CNN	Base classifiers are 5 CNNs. Each CNN has three convolutional layers, two pooling layers, a flatten layer, and a fully connected layer, using ReLU for hidden layers and Sigmoid for the output. BiLSTM is employed as the ensemble strategy. Parameters are based on Wang et al. ⁵² .
BiLSTM-Trans-CNN	Ensemble includes 5 CNN base classifiers, each with three convolutional layers, two pooling layers, a flatten layer, and a fully connected layer, using ReLU for hidden layers and Sigmoid for the output. Ensemble strategy combines BiLSTM and Transformer architectures. Parameters follow Wang et al. ⁵² .

Table 4. Main parameter settings for comparative models.

Hyperparameter	Description	Value
N_a	Attention layer dimension	{8, 16, 24, 32, 64}
N_d	Decision layer dimension	{8, 16, 24, 32, 64}
N_{step}	Number of decision steps	{3, 4, 5, 6},
γ	Relaxation factor that controls the mask	{1.0, 1.2, 1.5, 2.0}
μ	Control of the feature selection update process	{0.6, 0.7, 0.8, 0.9}

Table 5. Hyperparameters range for ECS TabNet.

	Actual positive	Actual negative
Predicted positive	TP	FN
Predicted negative	FP	TN

Table 6. Confusion matrix of customer credit scoring.

In the parameter setting for the ECS-SDE model, first, for ECS TabNet, this paper refers primarily to the research by Arik and Pfister³³. The Adam optimizer is used with a learning rate of 0.006, a batch size of 128, and 70 epochs. The ranges for some of the hyperparameters are shown in Table 5. To solve the data imbalance problem and achieve higher economic benefits, this paper employs a multi-objective optimization algorithm, with cost-saving (Save) and geometric mean indicators as optimization objectives. The optimization is performed using the default multi-objective algorithm from the Optuna library in Python. A sensitivity analysis of the optimal hyperparameter combinations is provided in Sect. 4.8. Then, for the parameter settings of the ECS GMDH neural network, this paper refers to the study by Lemke and Müller⁵³. The maximum number of layers for the network is set to 20, and the data division method is set to random. The reference function form is $y = w_0 + w_1x_1 + w_2x_2 + w_3x_1x_2$, with the remaining parameters kept at their default values. Additionally, considering that the ECS GMDH model complexity parameter λ and the number of ECS TabNet base classifiers M have a significant impact on the performance of the ECS-SDE model, this paper conducts a sensitivity analysis of these important parameters in Sect. 4.8. All experiments are run on a Windows 10 $\times$ 64 system equipped with an Intel(R) Core(TM) i5 processor. The experiments are conducted in Python 3.7, and the coding implementation uses the deep learning framework PyTorch and the GmdhPy library.

Evaluation metrics

Traditional classification frameworks evaluate models based on statistical metrics, which typically aim to minimize misclassifications under the assumption of equal misclassification costs. However, cost-sensitive classification methods provide a comprehensive evaluation of the model performance, rather than simply aiming for the highest classification accuracy. Therefore, this paper employs two different types of metrics: precision-oriented metrics, which include AUC-PR⁵⁴, AUC-ROC⁵⁵, Brier Score⁻ (BS⁻), and Brier Score⁺ (BS⁺)⁵⁶; and a cost-oriented metric, namely cost savings (Save)¹⁹. These five metrics provide a comprehensive evaluation of the model's performance. The confusion matrix for customer credit scoring is shown in Table 6.

TP represents the number of true positives, FN represents the number of false negatives, FP represents the number of false positives, and TN represents the number of true negatives.

(1) Save: In credit scoring, business needs are typically cost-driven. Therefore, this paper uses the Save metric to evaluate improvements in model performance from a cost-efficiency perspective. The Save metric¹⁹ is defined as the cost reduction achieved by using a model compared to not using any model. Specifically, Save assumes that all samples are predicted as the default class with the lowest cost (either 0 or 1), i.e., the baseline cost $C_{base} = \min\{C(y, 0), C(y, 1)\}$. It then calculates the total cost saved by the model's classification compared to C_{base} . The formula is as follows:

$$S(y, \hat{y}) = \frac{C_{base} - C(y, \hat{y})}{C_{base}} \tag{21}$$

when the model shows improvement in cost, the Save value lies between [0, 1], with the higher value indicating better performance.

(2) AUC-PR: The precision-recall (PR) curve shows the trade-off between precision and recall. Precision is the proportion of true positives among all samples predicted as positive, i.e., precision = $TP / (TP + FP)$, while recall is the proportion of actual positives correctly identified, i.e., recall = $TP / (TP + FN)$. This paper uses the area under the precision-recall curve (AUC-PR)⁵⁴ to assess the model's ability to discriminate positive samples, with a higher AUC-PR indicating better performance.

(3) AUC-ROC: The receiver operating characteristic curve (ROC) curve plots the true positive rate (TPR) against the false positive rate (FPR), where the x-axis is the false positive rate $FPR = FP / (FP + TN)$, and the y-axis is the true positive rate $TPR = TP / (TP + FN)$. It evaluates performance under uncertain class distributions or misclassification costs. The area under the ROC curve (AUC-ROC)⁵⁵ is used to assess performance, with higher values indicating better results.

(4) BS^+ : BS^+ is defined as the mean squared error of the minority class (positive class) samples, reflecting the model's calibration for the minority class. It is calculated as follows:

$$BS^+ = \frac{\sum_{i=1}^{N_{min}} (y_i^{min} - \hat{y}_i^{min})^2}{N_{min}} \quad (22)$$

where $\hat{y}_i^{min}$ is the predicted probability that sample i belongs to the minority class, y_i^{min} is the actual label of the minority-class sample, and N_{min} is the number of minority-class samples. Lower BS^+ values indicate better calibration for minority-class samples.

(5) BS^- : BS^- is defined as the mean squared error of the majority class (negative class) samples, indicating the model's calibration for the majority class. It is calculated as follows:

$$BS^- = \frac{\sum_{i=1}^{N_{maj}} (y_i^{maj} - \hat{y}_i^{maj})^2}{N_{maj}} \quad (23)$$

where $\hat{y}_i^{maj}$ is the predicted probability that sample i belongs to the majority class, y_i^{maj} is the actual label of the majority-class sample, and N_{maj} is the number of majority-class samples. Lower BS^- values indicate better calibration for the majority-class samples.

Dataset	Metrics	ECS-SDE	ECS-AdaBoost	ECSDNN	ECS-Stacking	CSNNE	CSCNN	CCS-CNN
GMSC	Save	0.45448(1) (± 0.01887)	0.16745(4) (± 0.04413)	0.06328(7) (± 0.01639)	0.11156(6) (± 0.03951)	0.12958(5) (± 0.01304)	0.42712(3) (± 0.04785)	0.43247(2) (± 0.03099)
	AUC-PR	0.24449(1) (± 0.01451)	0.14650(5) (± 0.01912)	0.07247(7) (± 0.02856)	0.16840(2) (± 0.01741)	0.14755(3) (± 0.01231)	0.13393(6) (± 0.01715)	0.14688(4) (± 0.03991)
	AUC-ROC	0.80102(1) (± 0.03872)	0.59442(5) (± 0.03445)	0.53987(7) (± 0.04943)	0.60414(4) (± 0.02218)	0.57558(6) (± 0.02932)	0.74163(2) (± 0.03827)	0.72027(3) (± 0.04221)
	BS^+	0.21223(2) (± 0.03133)	0.79336(6) (± 0.04671)	0.07813(1) (± 0.01342)	0.80456(7) (± 0.04831)	0.21348(3) (± 0.01567)	0.21471(4) (± 0.02408)	0.33831(5) (± 0.02766)
	BS^-	0.15019(2) (± 0.04304)	0.01781(1) (± 0.01586)	0.84722(7) (± 0.03085)	0.17215(3) (± 0.00408)	0.84037(6) (± 0.02549)	0.33202(5) (± 0.02765)	0.23432(4) (± 0.02105)
PAKDD	Save	0.30556(1) (± 0.03316)	0.01339(5) (± 0.04673)	0.29005(2) (± 0.04911)	0.03813(3) (± 0.02017)	0.02054(4) (± 0.02128)	-0.11614(6) (± 0.04646)	-0.11889(7) (± 0.04207)
	AUC-PR	0.25155(1) (± 0.02197)	0.21487(6) (± 0.02912)	0.21672(4) (± 0.02538)	0.21644(5) (± 0.01911)	0.20856(7) (± 0.01572)	0.22962(2) (± 0.05958)	0.22002(3) (± 0.03538)
	AUC-ROC	0.60951(1) (± 0.04021)	0.52709(5) (± 0.00493)	0.57090(4) (± 0.03564)	0.52303(6) (± 0.01309)	0.51343(7) (± 0.01753)	0.58100(3) (± 0.04997)	0.58927(2) (± 0.01519)
	BS^+	0.18299(1) (± 0.03758)	0.90097(6) (± 0.03379)	0.21574(4) (± 0.03252)	0.89527(5) (± 0.03883)	0.95974(7) (± 0.02071)	0.18311(2) (± 0.04641)	0.19402(3) (± 0.01912)
	BS^-	0.38199(3) (± 0.03635)	0.04486(2) (± 0.04358)	0.66701(5) (± 0.04822)	0.38868(4) (± 0.01290)	0.01339(1) (± 0.00587)	0.72689(7) (± 0.01221)	0.67742(6) (± 0.02802)
DCCC	Save	0.33307(1) (± 0.01751)	0.20668(6) (± 0.03777)	0.33114(2) (± 0.04156)	0.24954 (4) (± 0.02843)	0.22801(5) (± 0.02834)	0.26664(3) (± 0.01480)	0.17620(7) (± 0.13519)
	AUC-PR	0.41299(1) (± 0.01704)	0.33967(5) (± 0.03199)	0.23908(7) (± 0.02725)	0.36951(2) (± 0.03695)	0.35124(4) (± 0.01835)	0.32253(6) (± 0.01624)	0.35147(3) (± 0.01293)
	AUC-ROC	0.72550(1) (± 0.03672)	0.64068(5) (± 0.02462)	0.55405(7) (± 0.01565)	0.65855(3) (± 0.02602)	0.64598(4) (± 0.01790)	0.68318(2) (± 0.01816)	0.57631(6) (± 0.02448)
	BS^+	0.22276(1) (± 0.02338)	0.63053(6) (± 0.04016)	0.50299(4) (± 0.01193)	0.60312(5) (± 0.03546)	0.63110(7) (± 0.04192)	0.25766(3) (± 0.03573)	0.17699(2) (± 0.03593)
	BS^-	0.12378(3) (± 0.04118)	0.08812(2) (± 0.03901)	0.43332(5) (± 0.03520)	0.14978(4) (± 0.02494)	0.07695(1) (± 0.01423)	0.68063(7) (± 0.04561)	0.55013(6) (± 0.30796)
IEEE	Save	0.51258(1) (± 0.03616)	0.36019(6) (± 0.03650)	0.49914(3) (± 0.01945)	0.50924(2) (± 0.02937)	0.36484(5) (± 0.02510)	0.43393(4) (± 0.02430)	0.26378(7) (± 0.03994)
	AUC-PR	0.50040(1) (± 0.01391)	0.39409(4) (± 0.00922)	0.12108(7) (± 0.02830)	0.47504(3) (± 0.01777)	0.39388(5) (± 0.01533)	0.12829(6) (± 0.03059)	0.49760(2) (± 0.04237)
	AUC-ROC	0.86714(1) (± 0.03963)	0.73798(5) (± 0.00909)	0.77627(3) (± 0.01648)	0.74302(4) (± 0.01079)	0.71262(6) (± 0.01544)	0.79905(2) (± 0.03401)	0.43413(7) (± 0.04328)
	BS^+	0.36915(2) (± 0.04371)	0.46567(4) (± 0.03073)	0.48227(5) (± 0.01301)	0.51275(6) (± 0.02171)	0.57245(7) (± 0.03161)	0.25768(1) (± 0.03725)	0.43981(3) (± 0.04574)
	BS^-	0.01314(3) (± 0.01094)	0.00501(2) (± 0.01010)	0.10852(5) (± 0.03301)	0.04121(4) (± 0.01014)	0.00230(1) (± 0.01076)	0.14422(6) (± 0.01258)	0.30067(7) (± 0.02689)
Average ranking		1.45	4.50	4.80	4.10	4.70	3.95	4.50

Table 7. Comparison of credit scoring performance among the seven models.

Comparison experiments

Comparison of different cost sensitive models

This section compares the ECS-SDE model with three ECS models and three CCS models in terms of credit scoring performance. Table 7 shows the performance of the ECS-SDE model and six comparative models in the four datasets. In the table, bold indicates the top-performing model in each row, and the number in brackets indicates the model's ranking. The smaller the number, the better the model performance in credit scoring. In addition, the area in parentheses below the metric values represents the 95% confidence interval⁵⁷, which reflects the stability of the model's performance.

The results in Table 7 show that the ECS-SDE model consistently outperforms the other models, particularly excelling in the cost savings (Save) metric across all four datasets. Notably, the ECS-SDE model shows a 45.448% improvement in cost savings on the GMSC dataset and a 51.258% improvement on the IEEE dataset. This highlights the model's effectiveness in enhancing cost efficiency, optimizing resource allocation, and minimizing financial losses by accurately identifying high-risk customers and reducing the over-management of low-risk ones.

To further assess statistically significant differences between the seven models on each metric, this paper applies non-parametric statistical tests recommended by Demšar⁵⁸, namely the Friedman test⁵⁹ and the Iman-Davenport test⁶⁰. The null hypothesis for both tests is that the performance of the seven models is the same. For the 4 datasets and 7 models, we use a χ^2 distribution with 6 degrees of freedom and an F distribution with 6 and 18 (i.e., 6×3) degrees of freedom. The significance level is set at 0.05, with results presented in Table 8.

The test values exceed the corresponding distribution values, leading to the rejection of the null hypothesis at the 95% confidence level. This indicates significant performance differences between the seven models on each metric. Additionally, pairwise comparisons are conducted to further explore the performance differences among the models. First, we compute $z = (R_i - R_j) \sqrt{k(k-1)/(6 * Num)}$, where R_i and R_j are the average rankings of the i -th and j -th models, respectively, k is the number of models being compared (7 in this case), and Num is the number of datasets (4 in this case). After calculating z , it is converted into a probability value, and the Benjamini-Hochberg multiple testing correction⁶¹ is applied to obtain the adjusted p -values. Table 9 shows the results of the test.

From Table 9, it can be concluded that the ECS-SDE model shows significant advantages on multiple key metrics: (1) For the AUC-ROC metric, ECS-SDE shows a significant difference compared to ECS-AdaBoost, ECSDNN, ECS-Stacking, CSNNE, and CCS-CNN, with no significant difference observed between ECS-SDE and CSCNN. This indicates that ECS-SDE has stronger discriminatory power in the ROC curve area, allowing it to more accurately distinguish between high-risk and low-risk customers. (2) For the AUC-PR metric, ECS-SDE significantly outperforms ECS-AdaBoost, ECSDNN, CSNNE, and CSCNN models. This indicates that the ECS-SDE model has higher classification accuracy in handling class imbalance, particularly in identifying the minority-class samples (i.e. high-risk customers). (3) For the BS^+ metric, ECS-SDE significantly outperforms ECS-AdaBoost, ECS-Stacking, and CSNNE models. This highlights the efficacy of the ECS-SDE model in identifying positive samples and in detecting high-risk customers. (3) For the Save metric, ECS-SDE significantly outperforms ECS-AdaBoost, CSNNE, and CCS-CNN models, indicating superior performance in cost savings. (4) For the BS^- metric, ECS-SDE shows a significant advantage over CSCNN, despite its relatively average performance in predicting the negative class (low-risk customers). However, customer credit evaluation places more emphasis on the prediction of positive class samples, as accurately identifying high-risk customers is crucial for reducing financial losses. (6) Among the six comparison models, ECS-AdaBoost, ECSDNN, ECS-Stacking, CSNNE, CSCNN, and CCS-CNN show no significant performance differences across most metrics, indicating that their overall performance is similar.

In conclusion, the ECS-SDE model outperforms the six comparison models, particularly in handling class imbalance and identifying high-risk customers, with superior classification accuracy. The performance differences among the other models are minimal across most metrics, indicating their overall similarity.

Comparison of deep ensemble models

This section compares the performance of the ECS-SDE model with five advanced deep ensemble models in credit scoring (Table 10). To ensure fairness, we used the SMOTE technique¹² to generate new minority-class samples to balance the training set when training the deep ensemble comparative models. The area in parentheses below the metric values represents the 95% confidence interval⁵⁷. In the table, bold text highlights the top-performing model in each row.

The results in Table 10 show that the ECS-SDE model achieves the best overall average ranking among all comparison models, indicating that it has the best performance in credit scoring. It also outperforms other models in the cost savings (Save) metric across all four datasets, indicating its ability to accurately identify high-risk customers and reduce financial losses from misclassification.

To further analyze whether there are statistically significant differences between the ECS-SDE model and the five deep ensemble models in each metric, this paper still uses the Friedman test⁵⁹ and the Iman-Davenport

Test method	Save	AUC-PR	AUC-ROC	BS ⁺	BS ⁻
Friedman ($\chi^2_6 = 12.59$)	13.29	15.96	15.43	15.96	17.25
Iman-Davenport ($F_{(6,18)} = 2.66$)	3.72	5.96	5.40	5.96	7.67

Table 8. Results of the Friedman and Iman-Davenport tests for seven models.

Metrics	Models	ECS-AdaBoost	ECSDNN	ECS-Stacking	CSNNE	CSCNN	CCS-CNN
AUC-ROC	ECS-SDE	0.01748	0.01381	0.04907	0.00693	0.55683	0.03423
	ECS-AdaBoost		0.85011	0.70504	0.70504	0.09879	0.77972
	ECSDNN			0.67454	0.77972	0.07003	0.70504
	ECS-Stacking				0.49033	0.27420	0.85011
	CSNNE					0.03423	0.55683
	CSCNN						0.20760
AUC-PR	ECS-SDE	0.01748	0.00152	0.22850	0.02408	0.01748	0.22850
	ECS-AdaBoost		0.42581	0.22850	0.93959	1.00000	0.22850
	ECSDNN			0.04907	0.35957	0.42581	0.04907
	ECS-Stacking				0.27882	0.22850	1.00000
	CSNNE					0.93959	0.27882
	CSCNN						0.22850
BS ⁺	ECS-SDE	0.01748	0.21092	0.01381	0.01381	0.59022	0.27882
	ECS-AdaBoost		0.21092	0.85011	0.82303	0.08170	0.16986
	ECSDNN			0.16986	0.13716	0.59022	0.85011
	ECS-Stacking				0.85011	0.05888	0.13716
	CSNNE					0.04279	0.11291
	CSCNN						0.70504
Save	ECS-SDE	0.01381	0.17635	0.15807	0.03210	0.15807	0.00330
	ECS-AdaBoost		0.39034	0.44947	0.82303	0.44947	0.70504
	ECSDNN			0.89261	0.51706	0.89261	0.17635
	ECS-Stacking				0.59022	1.00000	0.20760
	CSNNE					0.59022	0.51706
	CSCNN						0.20760
BS ⁻	ECS-SDE	0.59022	0.08782	0.59022	0.74073	0.02853	0.06127
	ECS-AdaBoost		0.02408	0.22850	0.74073	0.01406	0.01748
	ECSDNN			0.30026	0.04206	0.70504	0.85011
	ECS-Stacking				0.38526	0.12344	0.22850
	CSNNE					0.01748	0.02853
	CSCNN						0.74073

Table 9. Results of the pairwise comparisons of seven models. Bold values indicate that the adjusted *p*-value is less than 0.05.

test⁶⁰. The null hypothesis for both tests is that the performance of the six models is the same. When the number of datasets is 4 and the number of models is 6, we use a χ^2 distribution with 5 degrees of freedom and an *F* distribution with 5 and 15 (5 × 3) degrees of freedom, with a significance level of 0.05. The test results are shown in Table 11.

As shown in Table 11, the test values are all greater than the corresponding distribution values. Therefore, at a 95% confidence level, we reject the null hypothesis and conclude that there are significant differences in the performance of the six models across each metric. To further understand the performance differences between the six models, we perform pairwise comparisons of the model performance. We also apply the Benjamini-Hochberg multiple testing correction⁶¹ to obtain the adjusted *p*-values. The results are shown in Table 12. In the table, bold values indicate that the adjusted *p*-value is less than 0.05.

According to Table 12, the ECS-SDE model shows significant advantages in most metrics: (1) For the Save metric, ECS-SDE significantly outperforms the LSTM-GRU-ANN, LSTM-GRU-MLP, BiLSTM-CNN, and BiLSTM-Trans-CNN models, but there is no significant difference when compared to CNN-BLSTM, indicating that ECS-SDE excels in cost savings. (2) ECS-SDE significantly outperforms the CNN-BLSTM, BiLSTM-CNN, and BiLSTM-Trans-CNN models, showing higher accuracy in identifying minority-class samples (high-risk customers). (3) For the AUC-ROC and AUC-PR metrics, ECS-SDE significantly outperforms the BiLSTM-CNN and BiLSTM-Trans-CNN models. (4) For the BS⁻ metric, ECS-SDE significantly outperforms the LSTM-GRU-ANN and BiLSTM-Trans-CNN models. (6) For LSTM-GRU-ANN, LSTM-GRU-MLP, CNN-BLSTM, BiLSTM-CNN, and BiLSTM-Trans-CNN models, no significant differences are observed in most metrics, indicating that their performance is relatively similar.

In conclusion, the ECS-SDE model excels in key metrics, particularly outperforming most deep ensemble models in Save and BS⁺ metrics. Its superior ability to identify high-risk customers and reduce financial losses highlights its effectiveness in cost savings and minority-class prediction.

Datasets	Metrics	ECS-SDE	LSTM-GRU-ANN	LSTM-GRU-MLP	CNN- BLSTM	BiLSTM-CNN	BiLSTM-Trans-CNN
GMSC	Save	0.45448(1) (± 0.01887)	0.45255 (3) (± 0.02754)	0.44879 (5) (± 0.02872)	0.45048 (4) (± 0.01435)	0.33459 (6) (± 0.03692)	0.45638 (2) (± 0.01839)
	AUC-PR	0.24449(1) (± 0.01451)	0.15741 (4) (± 0.01324)	0.15519 (5) (± 0.03435)	0.17215 (3) (± 0.02324)	0.18976 (2) (± 0.00619)	0.13061 (6) (± 0.01836)
	AUC-ROC	0.80102(1) (± 0.03872)	0.75952 (2) (± 0.03567)	0.75256 (3) (± 0.02446)	0.74638 (4) (± 0.01223)	0.67733 (6) (± 0.03597)	0.74343 (5) (± 0.01299)
	BS ⁺	0.21223 (1) (± 0.03133)	0.27363 (3) (± 0.04436)	0.29424 (4) (± 0.04136)	0.36674 (5) (± 0.02326)	0.54003 (6) (± 0.04525)	0.22046 (2) (± 0.04880)
	BS ⁻	0.15019 (1) (± 0.04304)	0.20732 (5) (± 0.04643)	0.20064 (4) (± 0.03216)	0.15951 (3) (± 0.02346)	0.15531 (2) (± 0.02334)	0.31267 (6) (± 0.04900)
PAKDD	Save	0.30556(1) (± 0.03316)	0.11141 (5) (± 0.01242)	0.15683 (4) (± 0.02436)	0.19230 (2) (± 0.03437)	0.14678 (6) (± 0.03504)	0.16268 (3) (± 0.04154)
	AUC-PR	0.25155(1) (± 0.02197)	0.24275 (4) (± 0.03448)	0.25110 (2) (± 0.009423)	0.25000 (3) (± 0.04154)	0.24029 (5) (± 0.01476)	0.23869 (6) (± 0.01394)
	AUC-ROC	0.60951(1) (± 0.04021)	0.58504 (4) (± 0.02547)	0.60083 (2) (± 0.03216)	0.59762 (3) (± 0.01346)	0.58203 (5) (± 0.00775)	0.57823 (6) (± 0.01637)
	BS ⁺	0.18299(1) (± 0.03758)	0.49906 (6) (± 0.04141)	0.47649 (4) (± 0.03221)	0.44828 (2) (± 0.03456)	0.46803 (3) (± 0.04274)	0.49530 (5) (± 0.01012)
	BS ⁻	0.38199 (3) (± 0.03635)	0.39007 (5) (± 0.02350)	0.33086 (2) (± 0.03222)	0.32827 (1) (± 0.02336)	0.39791 (6) (± 0.04960)	0.38824 (4) (± 0.00877)
DCCC	Save	0.33307(1) (± 0.01751)	0.30498 (4) (± 0.01772)	0.30975 (3) (± 0.03316)	0.32182 (2) (± 0.02408)	0.18496 (6) (± 0.03701)	0.20921 (5) (± 0.04914)
	AUC-PR	0.41299(1) (± 0.01704)	0.36568 (3) (± 0.022082)	0.35958 (4) (± 0.03879)	0.38707 (2) (± 0.03567)	0.34615 (5) (± 0.01274)	0.33523 (6) (± 0.04805)
	AUC-ROC	0.72550(1) (± 0.03672)	0.68748 (4) (± 0.03678)	0.69414 (3) (± 0.03213)	0.70537 (2) (± 0.01193)	0.65016 (6) (± 0.03955)	0.65645 (5) (± 0.04450)
	BS ⁺	0.22276(1) (± 0.02338)	0.45140 (3) (± 0.02662)	0.38660 (2) (± 0.01239)	0.52629 (5) (± 0.04272)	0.57824 (6) (± 0.04396)	0.49509 (4) (± 0.01365)
	BS ⁻	0.12378 (2) (± 0.04118)	0.35958 (6) (± 0.02126)	0.17363 (4) (± 0.03309)	0.10163 (1) (± 0.01723)	0.12944 (3) (± 0.04771)	0.19201 (5) (± 0.04931)
IEEE	Save	0.51258(1) (± 0.03616)	0.50320 (3) (± 0.03021)	0.50170 (4) (± 0.02301)	0.51007 (2) (± 0.01331)	0.41981 (5) (± 0.04353)	0.41855 (6) (± 0.02332)
	AUC-PR	0.50040(1) (± 0.01391)	0.23584 (2) (± 0.04931)	0.22872 (3) (± 0.02323)	0.18457 (4) (± 0.03351)	0.14851 (5) (± 0.01191)	0.13404 (6) (± 0.04439)
	AUC-ROC	0.86714(1) (± 0.03963)	0.83397 (3) (± 0.02911)	0.84520 (2) (± 0.01903)	0.82097 (4) (± 0.03421)	0.77659 (5) (± 0.02285)	0.59283 (6) (± 0.03782)
	BS ⁺	0.36915 (1) (± 0.04371)	0.36961 (2) (± 0.03211)	0.37029 (3) (± 0.04951)	0.37155 (4) (± 0.02361)	0.38498 (6) (± 0.04353)	0.37763 (5) (± 0.03430)
	BS ⁻	0.01314 (1) (± 0.01094)	0.08652 (4) (± 0.03343)	0.06074 (2) (± 0.01208)	0.06999 (3) (± 0.01991)	0.10719 (5) (± 0.01191)	0.11256 (6) (± 0.00584)
Average ranking		1.15	3.75	3.25	2.95	4.95	4.95

Table 10. Comparison of ECS-SDE with five deep ensemble models.

Test method	Save	AUC-PR	AUC-ROC	BS ⁺	BS ⁻
Friedman ($\chi^2_5 = 11.07$)	14.71	15.29	17.57	11.29	12.71
Iman-Davenport ($F_{(5,15)} = 2.90$)	8.35	9.73	21.71	3.89	5.24

Table 11. Results of the Friedman and Iman-Davenport tests for six models.

Computational time comparison of deep ensemble models

This section compares the computational time of the ECS-SDE model with five advanced deep ensemble models. Table 13 shows the time required by the six models to fit on the training set and make predictions on the test set. Bold text indicates the model with the shortest computation time in each row, with the number in brackets representing the model’s ranking, where a lower value indicates a shorter computation time. The last row presents the average ranking of total time for each model.

From Table 13, it can be seen that the average ranking of the ECS-SDE model is the same as that of the LSTM-GRU-ANN model, indicating that the computational time of the ECS-SDE model is at a moderate level among the six models. However, the computational time of the ECS-SDE model varies across different datasets. For instance, on the IEEE dataset, which has a large number of samples, high imbalance, and many features, the ECS-SDE model may require more complex processing, leading to increased computational time. On the other hand, on the PAKDD dataset, which has fewer samples and lower imbalance, the ECS-SDE model ranks 4th, with relatively shorter computational time compared to the LSTM-GRU-ANN and LSTM-GRU-MLP deep ensemble models. In contrast, the CNN-BLSTM model has the shortest overall computational time, with

Metrics	Models	LSTM-GRU-ANN	LSTM-GRU-MLP	CNN- BLSTM	BiLSTM-CNN	BiLSTM-Trans-CNN
Save	ECS-SDE	0.04172	0.02734	0.24506	0.00032	0.02734
	LSTM-GRU-ANN		0.88185	0.32944	0.18410	0.88185
	LSTM-GRU-MLP			0.24506	0.22036	1.00000
	CNN- BLSTM				0.02734	0.24506
	BiLSTM-CNN					0.22036
BS ⁺	ECS-SDE	0.09505	0.13251	0.03645	0.00216	0.03645
	LSTM-GRU-ANN		0.88185	0.75545	0.25184	0.75545
	LSTM-GRU-MLP			0.68500	0.18410	0.68500
	CNN- BLSTM				0.43925	1.00000
	BiLSTM-CNN					0.43925
AUC-ROC	ECS-SDE	0.06626	0.24506	0.06626	0.00043	0.00043
	LSTM-GRU-ANN		0.57962	1.00000	0.06626	0.06626
	LSTM-GRU-MLP			0.57962	0.02734	0.02734
	CNN- BLSTM				0.06626	0.06626
	BiLSTM-CNN					1.00000
AUC-PR	ECS-SDE	0.09465	0.06337	0.13807	0.02738	0.00012
	LSTM-GRU-ANN		0.82306	0.82306	0.50604	0.05215
	LSTM-GRU-MLP			0.75545	0.62792	0.06337
	CNN- BLSTM				0.39533	0.03645
	BiLSTM-CNN					0.19587
BS ⁻	ECS-SDE	0.01825	0.39533	0.82306	0.11043	0.01825
	LSTM-GRU-ANN		0.13807	0.02734	0.42818	0.82306
	LSTM-GRU-MLP			0.42818	0.42818	0.11043
	CNN- BLSTM				0.13807	0.01825
	BiLSTM-CNN					0.39533

Table 12. Pairwise comparison test results for six models.

Dataset	ECS-SDE	LSTM-GRU-ANN	LSTM-GRU-MLP	CNN- BLSTM	BiLSTM-CNN	BiLSTM-Trans-CNN
GMSC	5702.62 (5)	9242.66 (6)	5457.11 (4)	1442.35 (1)	1757.19 (3)	1523.67 (2)
PAKDD	1870.85 (4)	2956.66 (6)	1961.10 (5)	187.43 (1)	290.83 (3)	220.57 (2)
DCCC	1993.70 (6)	860.72 (4)	897.85 (5)	518.67 (2)	528.1 (1)	595.2 (3)
IEEE	53479.49 (6)	19844.16 (5)	13839.01 (4)	4831.90 (3)	2962.63 (2)	2905.3 (1)
Average ranking	5.25	5.25	4.50	1.75	2.25	2.00

Table 13. Comparison of computation time (unit: s).

an average ranking of 1.75, indicating it completes computations faster across multiple datasets. The average rankings of the BiLSTM-CNN and BiLSTM-Trans-CNN models are 2.25 and 2.00, respectively, with slightly longer computational times than the CNN-BLSTM model. The LSTM-GRU-MLP model has an average ranking of 4.50, with moderate computational time.

Ablation experiment

To analyze the impact of the ECS TabNet training process and the ECS GMDH selective ensemble process on the performance of the ECS-SDE model, we conducted an ablation experiment (Table 14). The experiment compared the credit scoring performance of three models on four datasets. The three models are as follows: (1) ECSTabNet+SRCGMDH selective deep ensemble model, which uses ECS TabNet as the base classifier and applies SRC-based GMDH for selective ensemble; (2) TabNet+ECSGMDH selective deep ensemble model, which uses the traditional TabNet as the base classifier and applies ECS GMDH for selective ensemble; (3) The proposed ECS-SDE model. In the table, bold text highlights the top-performing model in each row.

Table 14 shows that the ECS-SDE model, which combines the two techniques, has the highest average ranking, indicating the best performance in credit scoring. To further analyze whether there are statistically significant differences in the performance of the three models, we used the non-parametric Wilcoxon rank-sum test⁶². The null hypothesis is that the credit scoring performance of the two models is the same. We define R^+ as the sum of the ranks where the first model is better than the second, and R^- as the sum of the ranks where the first model is worse than the second. In this study, we set the significance level to $\alpha = 0.05$. At a 95% confidence level, when the number of data sizes is 20, the corresponding critical value (CV) is 52. The results of the rank-sum test comparing the performance of the three models are shown in Table 15. If $T = \min(R^+, R^-)$ is less

Datasets	Metrics	ECSTabNet + SRCGMDH	TabNet + ECSGMDH	ECS-SDE
GMS	Save	0.44036(2)	0.25329(3)	0.45448(1)
	AUC-PR	0.24319(2)	0.20808(3)	0.24449(1)
	AUC-ROC	0.80079(2)	0.63848(3)	0.80102(1)
	BS ⁺	0.19840(1)	0.27091(3)	0.21223(2)
	BS ⁻	0.15295(2)	0.15857(3)	0.14019(1)
PAKDD	Save	0.30444(2)	0.05202(3)	0.30556(1)
	AUC-PR	0.23855(3)	0.27665(1)	0.25155(2)
	AUC-ROC	0.59240(3)	0.59459(2)	0.60951(1)
	BS ⁺	0.18705(2)	0.42895(3)	0.18299(1)
	BS ⁻	0.41916(3)	0.15244(1)	0.38199(2)
DCCC	Save	0.33076(2)	0.30172(3)	0.33307(1)
	AUC-PR	0.43992(1)	0.41095(3)	0.41299(2)
	AUC-ROC	0.71824(2)	0.69484(3)	0.72550(1)
	BS ⁺	0.22586(2)	0.24352(3)	0.22276(1)
	BS ⁻	0.18231(3)	0.17478(2)	0.17378(1)
IEEE	Save	0.50054(2)	0.47631(3)	0.51258(1)
	AUC-PR	0.47742(3)	0.56474(1)	0.50040(2)
	AUC-ROC	0.85593(2)	0.81716(3)	0.86714(1)
	BS ⁺	0.37235(2)	0.37332(3)	0.36915(1)
	BS ⁻	0.01440(3)	0.01409(2)	0.01314(1)
Average ranking		2.20	2.55	1.25

Table 14. Credit scoring performance of the three models.

Comparison	$T = \min(R^+, R^-)$	CV	p-value	Hypothesis
ECS-SDE VS ECSTabNet + SRCGMDH	min (192.0, 18.0) = 18.0	52	0.000	Reject
ECS-SDE VS TabNet + ECSGMDH	min (196.5, 13.5) = 13.5	52	0.000	Reject
ECSTabNet + SRCGMDH VS TabNet + ECSGMDH	min (132.0, 78.0) = 78.0	52	0.287	Accept

Table 15. Results of the Wilcoxon rank sum test for the three models.

than or equal to 52, the null hypothesis is rejected, indicating a statistically significant difference between the two models. Specifically, if $T = R^-$ is less than or equal to 52, it means that the performance of the first model is statistically significantly better than the second model. Conversely, if $T = R^+$ is less than or equal to 52, the situation is reversed.

The results in Table 15 show that, at the 95% confidence level, the ECS-SDE model, which uses these two techniques, has statistically significantly better performance than the other two models. However, there is no significant difference in performance between the models that only use the ECS TabNet or the ECS GMDH. This suggests that the combination of the ECS TabNet with the ECS GMDH technique is critical to maximize the performance of the ECS SDE model.

Analysis of model interpretability

In practical scenarios, it is crucial not only to focus on model performance but also to analyze the impact of features on outcomes, especially for real-world applications like credit scoring. For instance, when a loan application is rejected, explaining the reasons to both the customer and manager is important. This section explores the interpretability of the proposed model, including visualizing the ECS GMDH selective ensemble process and analyzing the feature importance of ECS TabNet.

To explain the selection process of base classifiers, this paper visualizes the ECS GMDH network structure. According to the ECS GMDH selective ensemble modeling principle in Sect. 2.3, the prediction results of 20 base classifiers $\{T_1, T_2, \dots, T_{20}\}$ are used as the initial inputs $\{v_1, v_2, \dots, v_{20}\}$. These inputs are then combined pairwise through a transfer function $f(\cdot)$ to generate intermediate candidate models. The selection process follows the ECS-SC external criterion, where candidate models are chosen layer by layer based on the external criterion value. This process continues until the external criterion value reaches its minimum. The result is an optimal complexity model with a multilayer network structure. To visualize this, the selective ensemble process of ECS GMDH and the weight coefficients of each layer are presented.

This paper uses the GMS dataset as an example. Due to the large number of inputs at each layer, direct explanation is challenging. To simplify the ECS GMDH selective ensemble process, only the combination results of the selected base classifiers are retained (Fig. 4), with the corresponding weights listed in Table 16. By calculating layer by layer from back to front, an embedded polynomial combination function is finally

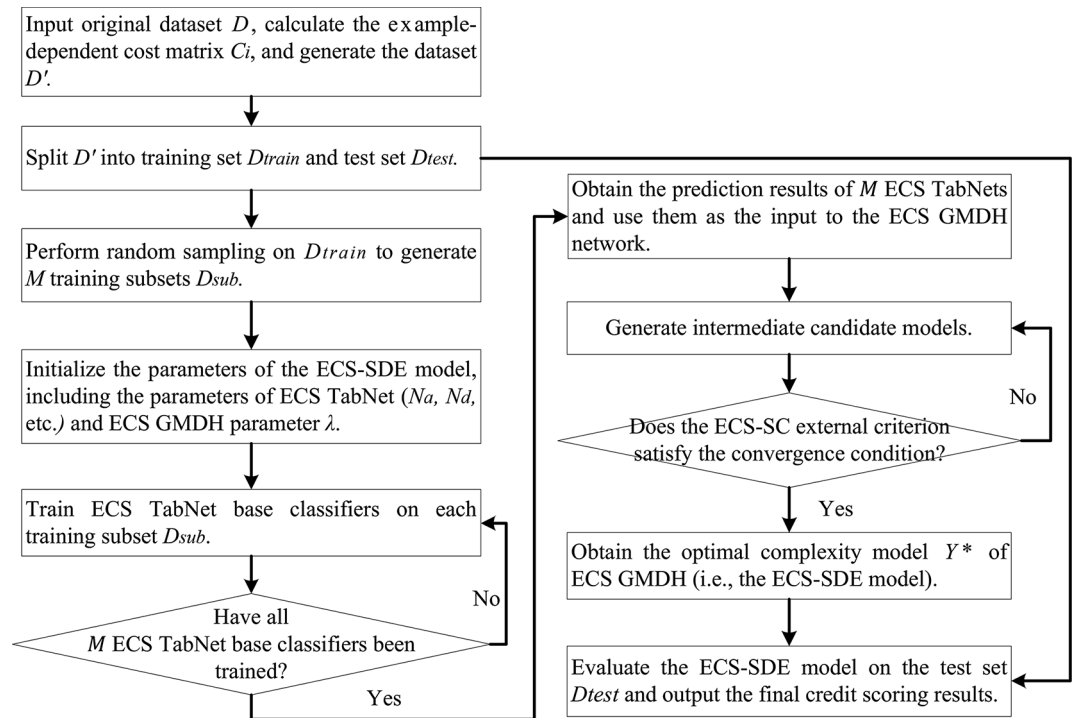


Fig. 3. Flowchart of ECS-SDE model.

Layer	Weight											
	Candidate model ₁				Candidate model ₂				Candidate model ₃			
	w_0	w_1	w_2	w_3	w_0	w_1	w_2	w_3	w_0	w_1	w_2	w_3
1	-0.02	1.05	-0.31	0.30	-0.20	0.86	0.42	0.05	-0.23	-0.64	0.59	-0.02
2	-2.07	4.16	0.50	-0.26	-1.57	2.43	-0.21	2.17	-3.27	4.85	3.26	-2.51
3	-2.52	2.59	2.98	-0.71	-2.13	4.24	0.33	-0.11				
4	-2.49	2.58	2.91	-0.71								
5	-2.29	4.59	-0.18	0.66								
6	-2.28	4.81	0.44	-1.03								
7	-2.33	4.71	-0.01	-0.13								
8	-2.30	5.04	0.38	-1.13								
9	-2.30	4.60	-0.18	0.53								
10	-2.25	4.62	0.31	-0.57								

Table 16. ECS GMDH network weights of each layer on the GMSC dataset.

obtained to represent the relationships among the selected optimal base classifiers. For example, in Layer 1, the combination of v_4 and v_9 is represented as $H_1 = f(v_4, v_9) = w_0 + w_1 v_4 + w_2 v_9 + w_3 v_4 v_9$. It is worth noting that the initial inputs $\{v_1, v_2, \dots, v_{20}\}$ are included in the candidate model set for each layer.

As shown in Fig. 4, on the GMSC dataset, the ECS GMDH model selects 10 optimal initial inputs ($v_1, v_2, v_4, v_8, v_9, v_{13}, v_{17}, v_{18}, v_{19}, v_{20}$), corresponding to the optimal ECS TabNet base classifiers: $T_1, T_2, T_4, T_8, T_9, T_{13}, T_{17}, T_{18}, T_{19}, T_{20}$. Table 16 shows the weights of each layer. Since the complexity of the polynomial functions and the large coefficients of individual terms compared to interaction terms, only the individual terms are retained, with interaction effects ignored. The simplified functional relationship between base classifiers and the prediction result on the GMSC dataset is as follows:

$$y = -1350.74T_1 + 113.72T_2 - 206.89T_4 - 52.71T_8 - 35.70T_9 - 38.71T_{13} - 2.23T_{17} - 269.81T_{18} + 39.36T_{19} - 473.24T_{20} \quad (24)$$

Based on the simplified function expression, we obtain the weights for the selected 10 base classifiers as $\{w_1 = -1350.74, w_2 = 113.72, \dots, w_{10} = -473.24\}$, and the influence of each base classifier on the prediction result: $T_1 > T_{20} > T_{18} > T_4 > T_2 > T_8 > T_{19} > T_{13} > T_9 > T_{17}$. According to the feature importance calculation method of TabNet described in Sect. 2.2, we calculate the importance score of each

base classifier for each feature. Let $S_k (k = 1, 2, \dots, 10) \in \mathbb{R}^d$ be the importance score of the k -th ECS TabNet base classifier for d features. The global importance of a feature reflects its contribution to the overall model performance³³. The feature importance scores $\{S_1, S_2, \dots, S_{10}\}$ output by the 10 selected base classifiers are summed and averaged to obtain the final global importance score for each feature: $S_{final} = \sum_{k=1}^{10} S_k / 10$.

Figure 5 presents the feature importance plot for the optimal ECS TabNet models selected by ECS GMDH on the GMSC dataset. Detailed feature descriptions are available in Appendix A. Feature importance plots and ECS GMDH selective ensemble results for the other three datasets can be found in Appendix B.

Figure 5 shows that, on the GMSC dataset, the top five most important features for the selected ECS TabNet classifiers ($T_1, T_2, T_4, T_8, T_9, T_{13}, T_{17}, T_{18}, T_{19}, T_{20}$) are: A_2 (Age), A_7 (Number of Times 90 Days Late), A_4 (Debt Ratio), A_9 (Number Of Time 60–89 Days Past Due Not Worse), and A_3 (Number Of Time 30–59 Days Past Due Not Worse). These features play a significant role in credit scoring prediction, as detailed below:

- Feature A_2 (Age) is generally considered an important factor in credit assessment. Older borrowers typically have more career experience and greater financial stability, which positively impacts their ability to repay loans. Therefore, age has a positive effect on credit scoring, especially when assessing a borrower's long-term repayment capacity.
- Feature A_4 (Debt Ratio) is a key indicator of a borrower's level of debt, representing the ratio of debt to income. A higher debt ratio typically signifies that the borrower is under more financial stress and has a weaker ability to repay debt, which increases credit risk. Therefore, A_4 is of significant reference value in credit assessment, particularly when evaluating whether a borrower has sufficient repayment capacity.
- Credit History Features, including A_3 (Number of Times 30–59 Days Past Due Not Worse), A_7 (Number of Times 90 Days Late), and A_9 (Number of Times 60–89 Days Past Due Not Worse). These features directly reflect the borrower's past repayment behavior. Multiple overdue records are generally seen as a sign of credit risk, as they indicate that the borrower may have had instability in repaying debts in the past. As such, these features help financial institutions better predict the borrower's future repayment behavior, influencing the approval of loan or credit card applications.

In summary, the five features mentioned above reflect key aspects of the borrower, such as repayment capacity, debt levels, and repayment history. Older age and lower debt ratios generally improve credit assessment, while

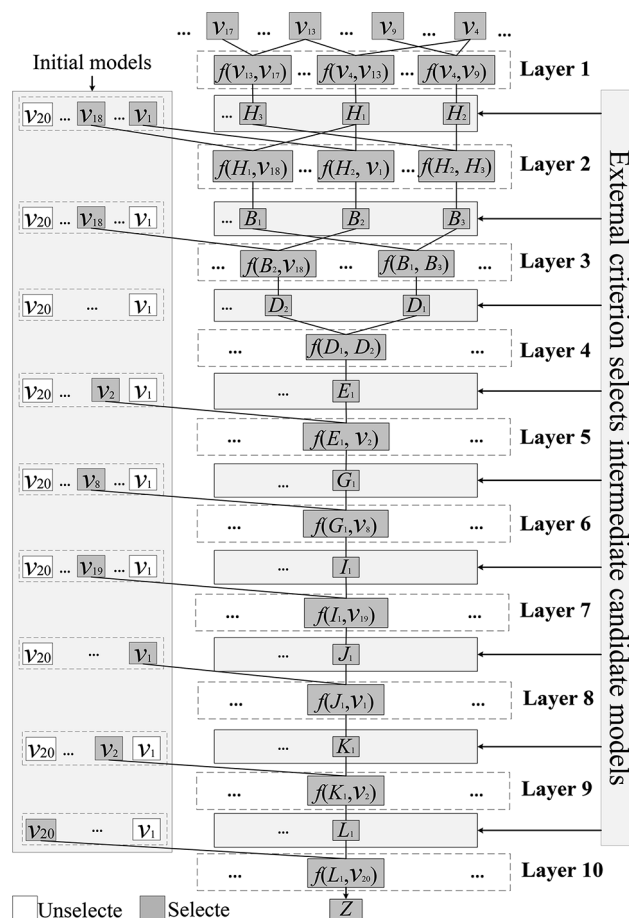


Fig. 4. Selective ensemble process of base classifiers on the GMSC dataset.

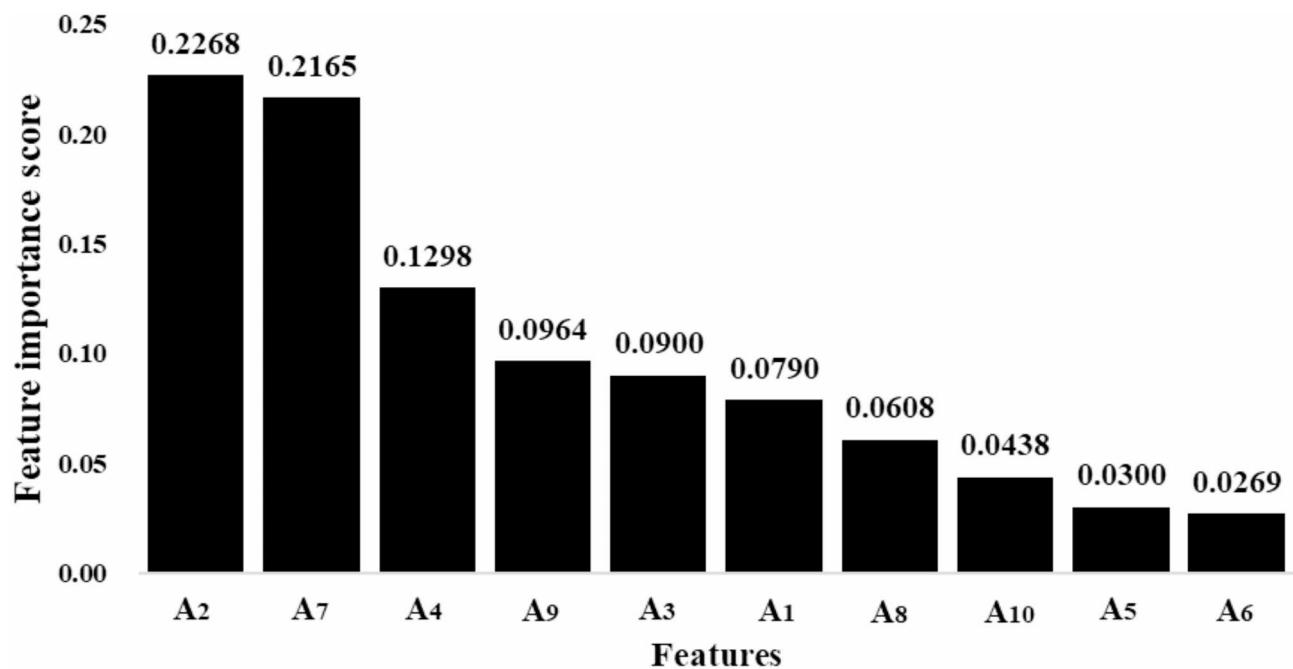


Fig. 5. Global feature importance plot on the GMSC dataset.

overdue records highlight past repayment behavior and credit risk, making these features essential for loan or credit card approval decisions.

In contrast, features like A_1 (Revolving Utilization of Unsecured Lines), A_5 (Monthly Income), A_6 (Number of Open Credit Lines and Loans), A_8 (Number of Real Estate Loans or Lines), and A_{10} (Number of Dependents) have a smaller impact on the model's predictions. While these features have limited influence, they still offer valuable insights into the borrower's financial situation. Financial institutions should consider these features alongside critical indicators, such as overdue records and debt ratio, for a more comprehensive and accurate risk assessment.

Parameter sensitivity analysis

In this section, the parameter sensitivity analysis is performed to investigate the effect of the parameters N_a , N_d , N_{step} , γ , and momentum of ECS TabNet on the performance of the ECS-SDE model. Additionally, the influence of the number of ECS TabNet base classifiers, M , on the performance of the ECS-SDE model in credit scoring is investigated. The impact of the complexity control parameter λ of the ECS GMDH on the performance of the ECS-SDE model is also analyzed. The results of the parameter sensitivity analysis are presented in Appendix C.

Conclusion

This paper proposes the ECS-SDE model and applies it to customer credit scoring. The model constructs an example-dependent cost matrix to generate ECS training subsets. It then integrates the proposed ECS TabNet and ECS GMDH deep neural networks to perform selective deep ensemble modeling. The experimental results show that the ECS-SDE model outperforms other comparison models in terms of overall performance for credit scoring. Notably, the ECS-SDE model shows strong interpretability, which reveals the importance of each feature in credit scoring. This interpretability analysis offers valuable insights for financial institutions to identify and mitigate customer default risk, enabling more precise risk management. In summary, the study provides an effective credit-scoring tool and has practical implications for improving deep learning model interpretability, ultimately reducing economic losses from customer defaults.

This paper offers key insights for financial institution management, including:

- (1) It is important to focus on core financial features, taking into account both personal information and financial status. (1) In credit scoring, financial institutions should prioritize core indicators like debt ratio and repayment history, as they directly reflect a borrower's repayment ability and credit risk. Higher debt ratios and overdue records should trigger closer scrutiny and prompt risk management measures, such as adjusting loan terms or conducting further risk assessments. (2) Institutions should adopt a comprehensive approach in borrower assessments, considering personal information (e.g., age, gender, occupation), financial status (e.g., income, debt ratio), and historical repayment records. This holistic evaluation enhances credit risk scoring and supports the development of more effective risk control strategies.
- (2) The improvement of the interpretability and transparency of the model is of great importance for the management of financial institutions. (1) Clear decision-making criteria enhance managers' understand-

ing of the model's process, fostering greater trust in its results. (2) Interpretable models enable managers to identify and manage potential risks, facilitating timely actions to sustain operations. (3) A transparent decision-making process ensures regulatory compliance, builds client trust, and supports more accurate business strategies.

Despite the promising potential of the ECS-SDE model in customer credit scoring, certain limitations remain. Future research could focus on the following areas: (1) Enhancing model interpretability. This study relies on feature correlation analysis, which may lead to biased or inconsistent interpretations when applied to complex business logic. Future research should incorporate causal inference techniques to better identify intrinsic feature relationships, enhancing both the accuracy and transparency of model behavior. (2) Optimizing computational resources. Training TabNet and GMDH models on large datasets is computationally intensive. Future research could address this limitation by investigating model compression, quantization, and knowledge distillation to improve efficiency and reduce hardware demands. (3) Expanding to multi-class scenarios. This study is limited to binary classification within the ECS framework. Future research could extend the ECS model to multi-class scenarios, addressing more complex, real-world applications.

Data availability

The datasets analyzed in the current study are publicly available from various sources. The IEEE-CIS Fraud Detection dataset can be accessed from the Kaggle competition (<https://www.kaggle.com/competitions/ieee-fraud-detection>). The Give Me Some Credit dataset is available at <http://www.kaggle.com/c/GiveMeSomeCredit/>. The Default of Credit Card Clients dataset is available through the UCI Machine Learning Repository (<https://archive.ics.uci.edu/ml/datasets/default+of+credit+card+clients>). Additionally, the 2009 Pacific-Asia Knowledge Discovery and Data Mining dataset can be accessed via <http://sede.neurotech.com.br:443/PAKDD2009/>. The GMDH library is available at <https://github.com/kvoyager/GmdhPy>. The TabNet library is available at <https://github.com/dreamquark-ai/tabnet>.

Received: 6 December 2024; Accepted: 10 February 2025

Published online: 18 February 2025

References

- Bressan, G., Đuranović, A., Monasterolo, I. & Battiston, S. Asset-level scoring of climate physical risk matters for adaptation finance. *Nat. Commun.* **15** (1), 5371 (2024).
- Petrone, D., Rodosthenous, N. & Latora, V. An AI approach for managing financial systemic risk via bank bailouts by taxpayers. *Nat. Commun.* **13** (1), 6815 (2022).
- Tang, Q., Tong, Z. & Yang, Y. Large portfolio losses in a turbulent market. *Eur. J. Oper. Res.* **292** (2), 755–769 (2021).
- Berger, L. M. et al. Inequality in high-cost borrowing and unemployment insurance generosity in US states during the COVID-19 pandemic. *Nat. Hum. Behav.* 1–13. <https://doi.org/10.1038/s41562-024-01922-8> (2024).
- Wang, Y. et al. Hyperspectral estimation of soil copper concentration based on improved TabNet model in the Eastern Junggar Coalfield. *IEEE Trans. Geosci. Remote Sens.* **60**, 1–20 (2022).
- Xiao, J. et al. Black-box attack-based security evaluation framework for credit card fraud detection models. *INFORMS J. Comput.* **35** (5), 986–1001 (2023).
- Xiao, J. et al. A novel deep ensemble model for imbalanced credit scoring in internet finance. *Int. J. Forecast.* **40** (1), 348–372 (2024).
- Bahnsen, A. C., Aouada, D. & Ottersten, B. Example-dependent cost-sensitive decision trees. *Expert Syst. Appl.* **42** (19), 6609–6619 (2015).
- Höppner, S., Baesens, B., Verbeke, W. & Verdonck, T. Instance-dependent cost-sensitive learning for detecting transfer fraud. *Eur. J. Oper. Res.* **297** (1), 291–300 (2022).
- Yotsawat, W., Wattuya, P. & Srivihok, A. A novel method for credit scoring based on cost-sensitive neural network ensemble. *IEEE Access*. **9**, 78521–78537 (2021).
- Zhao, H. et al. An ensemble learning approach with gradient resampling for class-imbalance problems. *INFORMS J. Comput.* **35** (4), 747–763 (2023).
- Almhaithawi, D., Jafar, A. & Aljndi, M. Example-dependent cost-sensitive credit cards fraud detection using SMOTE and Bayes minimum risk. *SN Appl. Sci.* **2** (9), 1–12 (2020).
- Janssens, B., Bogaert, M. & Bagué, A. & Van Den Poel, D. B2Boost: Instance-dependent profit-driven modelling of B2B churn. *Ann. Oper. Res.* **341**, 1–27 (2022).
- Vanderschuere, T., Verdonck, T., Baesens, B. & Verbeke, W. Predict-then-optimize or predict-and-optimize? An empirical evaluation of cost-sensitive learning strategies. *Inf. Sci.* **594**, 400–415 (2022).
- Lenarcik, A. & Piasta, Z. Rough classifiers sensitive to costs varying from object to object. *Proc. Int. Conf. Rough Sets Curr. Trends Comput.*, 222–230 (1998).
- Bahnsen, A. C., Aouada, D. & Ottersten, B. A novel cost-sensitive framework for customer churn predictive modeling. *Decis. Anal.* **2** (1), 1–15 (2015).
- Zadrozny, B., Langford, J. & Abe, N. Cost-sensitive learning by cost-proportionate example weighting. *Proc. 3rd IEEE Int. Conf. Data Min.*, 435–442 (2003).
- Elkan, C. The foundations of cost-sensitive learning. *Proc. Int. Joint Conf. Artif. Intell.* **17**, 973–978 (2001).
- Bahnsen, A. C., Aouada, D. & Ottersten, B. Example-dependent cost-sensitive logistic regression for credit scoring. *Proc. Int. Conf. Mach. Learn. Appl. (IEEE)*, 263–269 (2014).
- González, P. et al. Multiclass support vector machines with example-dependent costs applied to plankton biomass estimation. *IEEE Trans. Neural Netw. Learn. Syst.* **24** (11), 1901–1905 (2013).
- Bahnsen, A. C., Aouada, D. & Ottersten, B. Example-dependent cost-sensitive credit scoring using Bayes minimum risk. *Proc. Int. Conf. Mach. Learn. Appl. (IEEE)*, 10 (2014).
- Bahnsen, A. C., Stojanovic, A., Aouada, D. & Ottersten, B. Cost sensitive credit card fraud detection using Bayes minimum risk. *Proc. 12th Int. Conf. Mach. Learn. Appl. (IEEE)*, 1, 333–338 (2013).
- Bahnsen, A. C., Aouada, D. & Ottersten, B. Ensemble of example-dependent cost-sensitive decision trees. *Preprint Submitted May 18*, 6609 (2015). <https://arxiv.org/abs/1505.04637>
- Zelenkov, Y. Example-dependent cost-sensitive adaptive boosting. *Expert Syst. Appl.* **135**, 71–82 (2019).
- Bhargava, S. et al. A novel example-dependent cost-sensitive stacking classifier to identify tax return defaulters. *Proc. Bus. Inf. Syst.*, 343–353 (2021).

26. Bhuvaneshwari, K., Kannimuthu, S., Bhanu, D., Karthi, M. & Sagar, K. H. Effective radical driver support system using machine learning methods for connected vehicles. *Turk. J. Physiother Rehabil.* **32** (2), 1024–1031 (2020).
27. Saqr, A. E. S., Elshewey, A. M., Raju, S. K. & Eid, M. M. A comprehensive review on optimizing machine learning models for early detection and forecasting of monkeypox outbreaks. *J. Artif. Intell. Metaheuristics.* **8** (1), 9–20 (2024).
28. Zuo, C., Zhang, X., Yan, L. & Zhang, Z. G. U. G. E. N. Global user graph enhanced network for next POI recommendation. *IEEE Trans. Mob. Comput.* **23** (12), 14975–14986 (2024).
29. Zhu, C. Research on emotion recognition-based smart assistant system: emotional intelligence and personalized services. *J. Syst. Manag. Sci.* **13** (5), 227–242 (2023).
30. Peng, Y. et al. Unveiling user identity across social media: a novel unsupervised gradient semantic model for accurate and efficient user alignment. *Complex. Intell. Syst.* **11** (1), 1–28 (2025).
31. Li, T., Li, Y., Zhang, M., Tarkoma, S. & Hui, P. You are how you use apps: user profiling based on spatiotemporal app usage behavior. *ACM Trans. Intell. Syst. Technol.* **14** (4), 1–21 (2023).
32. Mehta, P., Babu, C. S., Rao, S. K. V., Kumar, S. & DeepCatch Predicting return defaulters in taxation system using example-dependent cost-sensitive deep neural networks. *Proc. IEEE Int. Conf. Big Data (IEEE)*, 4412–4419 (2020).
33. Arik, S., Ö. & Pfister (ed, T.) TabNet: attentive interpretable tabular learning. *Proc. AAAI Conf. Artif. Intell.* **35** 6679–6687 (2021).
34. Cai, Q. & He, J. Credit payment fraud detection model based on TabNet and Xgboost. *Proc. 2nd Int. Conf. Consum. Electron. Comput. Eng.* (IEEE), 823–826 (2022).
35. Zhang, L., Ma, K., Yuan, F. & Fang, W. A TabNet based card fraud detection algorithm with feature engineering. *Proc. 2nd Int. Conf. Consum. Electron. Comput. Eng.* (IEEE), 911–914 (2022).
36. Lee, W., Lee, S. & Seok, J. Credit card default prediction by using heterogeneous ensemble. *Proc. 14th Int. Conf. Ubiquitous Fut. Networks*, 907–910 (2023).
37. Geng, Y. & Luo, X. Cost-sensitive convolution based neural networks for imbalanced time-series classification. *Intell. Data Anal.* **23** (2), 357–370 (2019).
38. Vimala, G. A. G. et al. R. Cost sensitive learning using chest X-ray with CNN for Covid-19 detection with lung diseases leading to class imbalance. *In Proc. 5th Int. Conf. Image Process. Capsule Net.* (IEEE), 489–495 (2024).
39. Boughorbel, S., Jarray, F. & Kadri, A. Fairness in TabNet model by disentangled representation for the prediction of hospital no-show. *Preprint Submitted Mar. 6*, 2103.04048 (2021). <https://arxiv.org/abs/>
40. Joseph, L. P., Joseph, E. A. & Prasad, R. Explainable diabetes classification using hybrid bayesian-optimized TabNet architecture. *Comput. Biol. Med.* **151**, 106178 (2022).
41. McDonnell, K., Murphy, F., Sheehan, B., Masello, L. & Castignani, G. Deep learning in insurance: accuracy and model interpretability using TabNet. *Expert Syst. Appl.* **217**, 119543 (2023).
42. Ivakhnenko, A. G. The group method of data of handling: a rival of the method of stochastic approximation. *Sov Autom. Control.* **13**, 43–55 (1968).
43. Stepashko, V., Bulgakova, O. & Zosimov, V. Performance of hybrid multilayered GMDH algorithm *Proc. 4th Int. Workshop on Inductive Modelling (IWIM)*, 5–9 (2011).
44. Ivakhnenko, A., Ivakhnenko, G. & Muller, J. Self-organization of neural networks with active neurons. *Pattern Recognit. Image Anal.* **4** (2), 185–196 (1994).
45. Xiao, J. et al. Cost-sensitive semi-supervised selective ensemble model for customer credit scoring. *Knowl. -Based Syst.* **189**, 105118 (2020).
46. Wakitani, S. & Yamamoto, T. Study on a GMDH-PID controller design method based on LASSO. *Proc. 57th Annu. Conf. Soc. Instrum. Control Eng. Jpn.* (IEEE), 1464–1469 (2018).
47. Bahnsen, A. C. *Example-dependent cost-sensitive Classification with Applications in Financial risk Modeling and Marketing Analytics* (University of Luxembourg, 2015).
48. Yu, L., Yang, Z. & Tang, L. A novel multistage deep belief network based extreme learning machine ensemble learning paradigm for credit risk scoring. *Flex. Serv. Manuf. J.* **28**, 576–592 (2016).
49. Forough, J. & Momtazi, S. Ensemble of deep sequential models for credit card fraud detection. *Appl. Soft Comput.* **99**, 106883 (2021).
50. Mienye, I. D. & Sun, Y. A deep learning ensemble with data resampling for credit card fraud detection. *IEEE Access.* **11**, 30628–30638 (2023).
51. Haghighi, F. & Omranpour, H. Stacking ensemble model of deep learning and its application to Persian/Arabic handwritten digits recognition. *Knowl. -Based Syst.* **220**, 106940 (2021).
52. Wang, M., Ma, H., Wang, Y. & Sun, X. Design of smart home system speech emotion recognition model based on ensemble deep learning and feature fusion. *Appl. Acoust.* **218**, 109886 (2024).
53. Lemke, F. & Müller, J. A. Self-organizing data mining. *Syst. Anal. Model. Simul.* **43** (2), 231–240 (2003).
54. Boyd, K., Eng, K. H. & Page, C. D. Area under the precision-recall curve: Point estimates and confidence intervals. *Proc. ECML PKDD Conf.* 451–466 (2013).
55. Bradley, A. P. The use of the area under the ROC curve in the evaluation of machine learning algorithms. *Pattern Recogn.* **30** (7), 1145–1159 (1997).
56. Wallace, B. C. & Dahabreh, I. J. Improving class probability estimates for imbalanced data. *Knowl. Inf. Syst.* **41** (1), 33–52 (2014).
57. Student. The probable error of a mean. *Biometrika* **6** (1), 1–25 (1908).
58. Demšar, J. Statistical comparisons of classifiers over multiple data sets. *J. Mach. Learn. Res.* **7** (Jan), 1–30 (2006).
59. Friedman, M. A comparison of alternative tests of significance for the problem of m rankings. *Ann. Math. Stat.* **11** (1), 86–92 (1940).
60. Iman, R. L. & Davenport, J. M. Approximations of the critical region of the fbietkan statistic. *Commun. Stat. - Theory Methods.* **9** (6), 571–595 (1980).
61. Benjamini, Y. & Hochberg, Y. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *J. R Stat. Soc. B.* **57** (1), 289–300 (1995).
62. Wilcoxon, F. Individual comparisons by ranking methods. *Breakthroughs Stat.* 196–202 (1992).

Acknowledgements

This work is supported in part by the National Natural Science Foundation of China 72171160; 71988101; 72401208), the National Social Science Fund of China (24VRC096), the Postdoctoral Fellowship Program of CPSF (GZB20240504), the EU Horizon 2020 RISE Project ULTRACEPT under Grant (778062), Sichuan University Interdisciplinary Innovation Fund.

Author contributions

J.X.: Conceptualization; Methodology. S.L.: Data curation; Software; Writing-original draft. Y.T.: Software; Writing-review and Editing. J.H.: Supervision; Writing-reviewing. X.J.: Software; Writing-reviewing. S.W.: Supervision; Writing-reviewing.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Supplementary Information The online version contains supplementary material available at <https://doi.org/10.1038/s41598-025-89880-7>.

Correspondence and requests for materials should be addressed to S.L. or S.W.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025